

Chapter 10

OOP: Inheritance and Runtime Polymorphism

Learning Objectives

- Use traditional and modern inheritance idioms, and understand base classes and derived classes.
- Understand the order in which C++ calls constructors and destructors in inheritance hierarchies.
- See how runtime polymorphism can make programming more convenient and systems more easily extensible.
- Use `override` to tell the compiler that a derived-class function overrides a base-class virtual function.
- Use `final` at the end of a function's prototype to indicate that function may not be overridden.
- Use `final` after a class's name in its definition to indicate that a class cannot be a base class.
- Perform inheritance with abstract and concrete classes.
- See how C++ can implement virtual functions and dynamic binding—and get a sense of virtual function overhead.
- Use interfaces to create more flexible runtime-polymorphic systems.

Introduction

- This chapter continues our object-oriented programming (OOP) discussion by introducing **inheritance** and **runtime polymorphism**.
 - With inheritance, you'll create classes that absorb existing classes' capabilities, then customize or enhance them.
- When creating a class, you can specify that the new class should inherit an existing class's members.
 - This existing class is called the **base class**, and the new class is called the **derived class**.
 - Some programming languages, such as Java and C#, use the terms **superclass** and **subclass** for base class and derived class.

Introduction (Cont.)

- Has-a vs. Is-a Relationships

- We distinguish between the has-a relationship and the is-a relationship:

- ◆ The **has-a relationship represents composition** in which an object contains one or more objects of other classes as members.

- For example, a Car has a steering wheel, has a brake pedal, has an engine, has a transmission, etc.

- ◆ The **is-a relationship represents inheritance**.

- In an is-a relationship, a derived-class object also can be treated as an object of its base-class type.
 - For example, a Car is a Vehicle, so a Car also exhibits a Vehicle's behaviors and attributes.

Introduction (Cont.)

- Runtime Polymorphism

- We'll explain and demonstrate runtime polymorphism with inheritance hierarchies.
- Runtime polymorphism enables you to conveniently “program in the general” rather than “program in the specific.”
- Programs can process objects of classes related by inheritance as if they're all objects of the base-class type.
 - ◆ You'll see that runtime polymorphic code—as we'll initially implement it with inheritance and virtual functions—refers to objects via base-class pointers or base-class references.

Introduction (Cont.)

- Implementing for Extensibility

- With runtime polymorphism, you can design and implement more **easily extensible systems**.
- New classes can be added with little or no modification to the program's general portions as long as the new classes are part of the program's inheritance hierarchy.
- You **modify only code that requires direct knowledge of the new classes**.
 - ◆ Suppose a new class Car inherits from class Vehicle.
 - ◆ We need to write only the new class and code that creates Car objects and adds them to the system—the new class simply “plugs right in.”
 - ◆ The general code that processes Vehicles can remain the same.

Introduction (Cont.)

- Discussion of Runtime Polymorphism with Virtual Functions “Under the Hood”
 - A key feature of this chapter is its discussion of [runtime polymorphism](#), [virtual functions](#) and [dynamic binding](#) “under the hood.”
 - ◆ The C++ standard does not specify how language features should be implemented.
 - ◆ We use a detailed illustration to explain how runtime polymorphism with virtual functions can be implemented in C++.
 - This chapter presents a traditional introduction to inheritance and runtime polymorphism to acquaint you with basic- through intermediate-level thinking and ensure that you understand the mechanics of how these technologies work.
 - ◆ Later, we’ll address current thinking and programming idioms.

Introduction (Cont.)

- Interfaces and Dependency Injection

- To explain inheritance mechanics, our first few examples focus on **implementation inheritance**, which is primarily used to **define closely related classes** with many of **the same data members** and **member-function implementations**.
- For decades, implementation inheritance was widely practiced in the object-oriented programming community.
 - ◆ Over time, though, experience revealed its weaknesses.
- C++ is used to build real-world, business-critical and mission-critical systems—often at a grand scale.
- The empirical results from years of building such implementation-inheritance-based systems show that they can be challenging to maintain and modify.

Introduction (Cont.)

- Interfaces and Dependency Injection

- So, we'll refactor one of our implementation inheritance examples to use interface inheritance.
 - ◆ The base class will not provide any implementation details.
 - ◆ Instead, it will contain “placeholders” that tell derived classes the functions they're required to implement.
 - ◆ The **derived classes will provide the implementation details**—the data members and member-function implementations.
 - ◆ As part of this example, we'll introduce dependency injection, in which a class contains a pointer to an object that provides a behavior required by objects of the class—in our example, calculating an employee's earnings.
 - ◆ This pointer can be re-aimed—for example, if an employee gets promoted, we can change the earnings calculation by aiming the pointer at an appropriate object. Then, we'll discuss how this refactored example is easier to evolve.

Introduction.

- Chapter Goal

- Our goal in this chapter is to familiarize you with inheritance and runtime polymorphism mechanics.
 - ◆ You'll be able to better appreciate **modern compile-time polymorphism** approaches that can promote ease of modifiability and better performance.
- You'll see some of these approaches in Chapter 15, Templates, C++20 Concepts and Metaprogramming, and Chapter 20, Other Topics and a Look Toward the Future of C++. In Chapter 20, we'll discuss
 - ◆ non-virtual interface idiom (NVI),
 - ◆ runtime polymorphism via `std::variant` and `std::visit`, and
 - ◆ additional inheritance-related topics.

Base Classes and Derived Classes

- The following table lists several simple base-class and derived-class examples.

Base class	Derived classes
Student	GraduateStudent, UndergraduateStudent
Shape	Circle, Triangle, Rectangle, Sphere, Cube
Loan	CarLoan, HomeImprovementLoan, MortgageLoan
Vehicle	Car, Motorcycle, Boat
Employee	Faculty, Staff
Account	CheckingAccount, SavingsAccount

Base Classes and Derived Classes (Cont.)

- Base classes tend to be more general, and derived classes more specific.
- Every derived-class object is an object of its base-class type, and one base class can have many derived classes.
- So, the set of objects a base class represents is typically larger than the set of objects a derived class represents.

Base Classes and Derived Classes (Cont.)

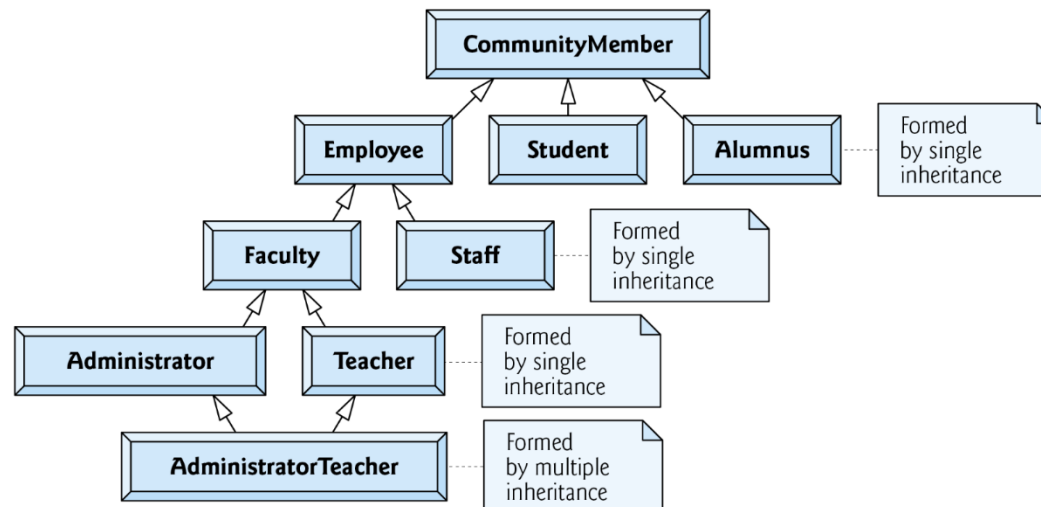
- CommunityMember Class Hierarchy

- Inheritance relationships naturally form class hierarchies.
 - ◆ A base class exists in a hierarchical relationship with its derived classes.
 - ◆ Once classes are employed in inheritance hierarchies, they become coupled with other classes.
- A class can be
 - ◆ a base class that **supplies members** to other classes,
 - ◆ a derived class that **inherits members** from other classes or
 - ◆ both.

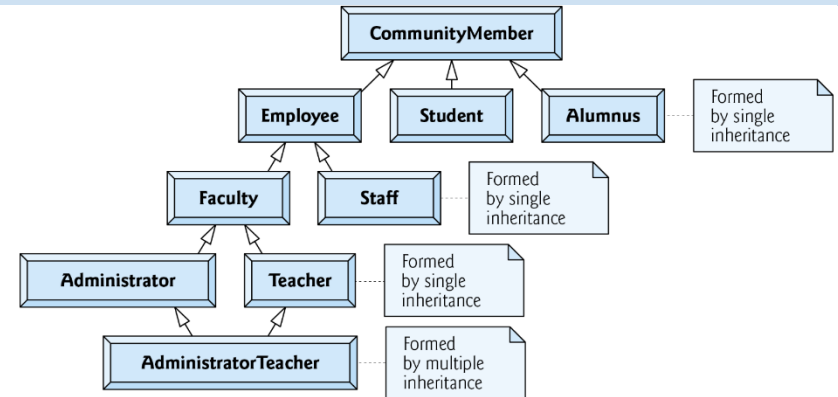
Base Classes and Derived Classes (Cont.)

- CommunityMember Class Hierarchy

- Let's develop the simple inheritance hierarchy represented by the following UML class diagram.
- Such diagrams illustrate the relationships among classes in a hierarchy:



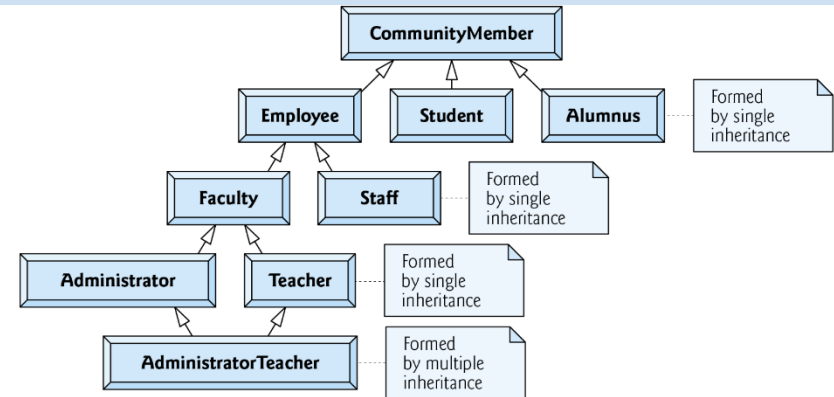
Base Classes and D (Cont.)



- CommunityMember Class Hi

- A university community might have thousands of CommunityMembers, such as Employees, Students and alumni (each of class Alumnus).
- Employees are either Faculty or Staff, and Faculty are either Administrators or Teachers.
- Some Administrators are also Teachers.
- With **single inheritance**, a derived class inherits from one base class.
- With **multiple inheritance**, a derived class inherits from two or more base classes. In this hierarchy, we've used multiple inheritance to form class AdministratorTeacher.

Base Classes and D (Cont.)

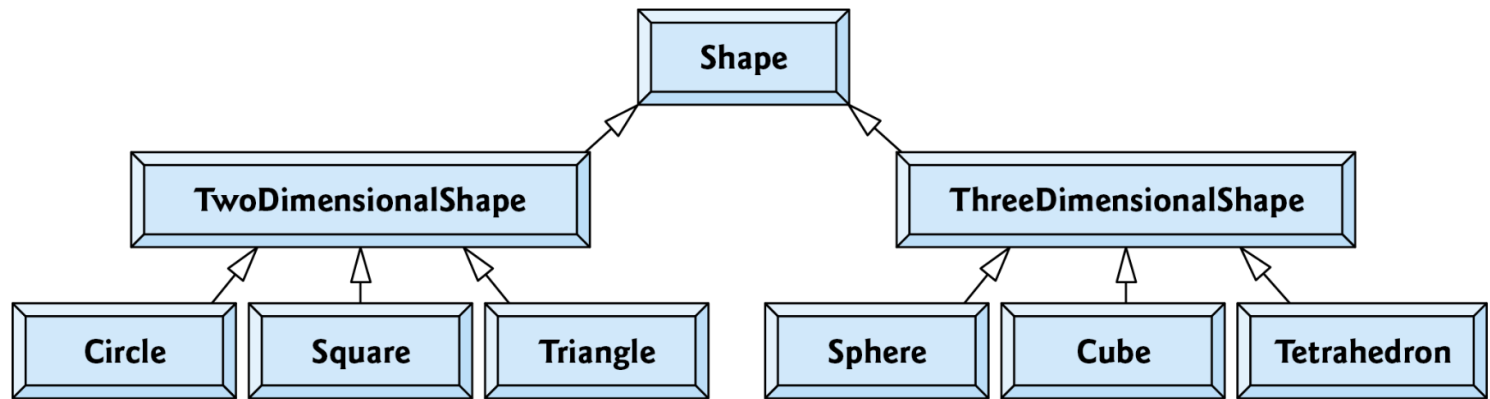


- CommunityMember Class Hi

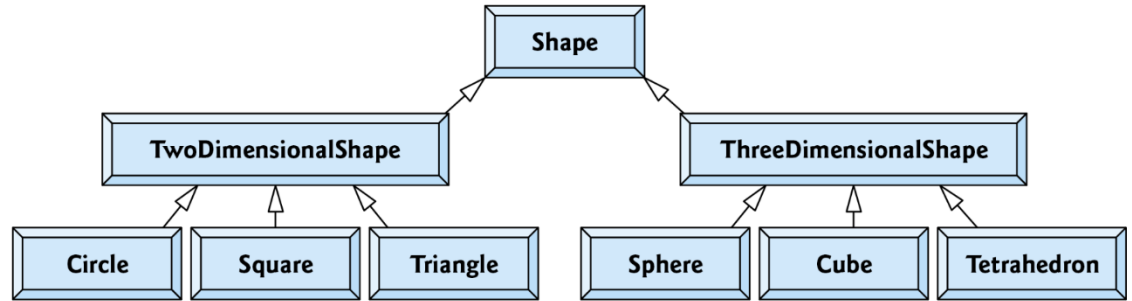
- Each upward-pointing arrow in the diagram represents an is-a relationship.
 - ◆ Following the arrows upward, we can state “an Employee is a CommunityMember” and “a Teacher is a Faculty member.”
 - ◆ CommunityMember is the direct base class of Employee, Student and Alumnus.
 - ◆ CommunityMember is an indirect base class of all the hierarchy’s other classes.
 - ◆ An indirect base class is two or more levels up the class hierarchy from its derived classes.
- You can follow the arrows upward several levels, applying the is-a relationship.
 - ◆ So, an AdministratorTeacher is an Administrator (and also is a Teacher), is a Faculty member, is an Employee and is a CommunityMember.

Base Classes and Derived Classes (Cont.)

- Shape Class Hierarchy and public Inheritance
 - Let's consider a Shape inheritance hierarchy:



Base Classes (Cont.)



- Shape Class Hierarchy and public Inheritance
 - Classes TwoDimensionalShape and ThreeDimensionalShape derive from base class Shape, so
 - ◆ a TwoDimensionalShape is a Shape, and
 - ◆ a ThreeDimensionalShape is a Shape.
 - The hierarchy's third level contains specific TwoDimensionalShapes and ThreeDimensionalShapes.
 - We can follow the arrows upward to identify direct and indirect is-a relationships. For instance,
 - ◆ a Triangle is a TwoDimensionalShape and is a Shape, while
 - ◆ a Sphere is a ThreeDimensionalShape and is a Shape.

Base Classes and Derived Classes (Cont.)

- Shape Class Hierarchy and public Inheritance
 - The following class header specifies that TwoDimensionalShape inherits from Shape:
class TwoDimensionalShape : public Shape
 - ◆ This is **public inheritance**, which we'll use in this chapter.
 - ◆ In public inheritance, public base-class members become public derived-class members and protected base-class members become protected derived-class members.
 - ◆ Although **private base-class members are not directly accessible in derived classes**, these members are still inherited and part of the derived-class objects.
 - ◆ The derived class can manipulate private base-class members through inherited base-class public and protected member functions—if they provide such functionality.
 - ◆ We'll discuss private and protected inheritance in Chapter 20.

Base Classes and Derived Classes.

- Shape Class Hierarchy and public Inheritance
 - Inheritance is not appropriate for every class relationship.
 - Composition's has-a relationship sometimes is more appropriate.
 - ◆ For example, given Employee, BirthDate and PhoneNumber classes, it's improper to say that an Employee is a BirthDate or that an Employee is a PhoneNumber.
 - ◆ However, an Employee has a BirthDate and has a PhoneNumber.
 - It's possible to treat base-class objects and derived-class objects similarly—their commonalities are expressed in the base-class members.
 - ◆ Later in this chapter, we'll consider examples that take advantage of that relationship.

Relationship Between Base and Derived Classes

- This section uses an inheritance hierarchy of employee types in a company's payroll application to demonstrate the relationship between base and derived classes:
 - Base-class salaried employees are paid a fixed weekly salary.
 - Derived-class salaried commission employees receive a weekly salary plus a percentage of their sales.

Relationship Between Base and Derived Classes (Cont.)

- Creating and Using a SalariedEmployee Class
 - Let's examine SalariedEmployee's class definition.

```
1 // SalariedEmployee.h
2 // SalariedEmployee class definition.
3 #pragma once // prevent multiple inclusions of header
4 #include <string>
5 #include <string_view>
6
7 class SalariedEmployee {
8 public:
9     SalariedEmployee(std::string_view name, double salary);
10
11     void setName(std::string_view name);
12     std::string getName() const;
13
14     void setSalary(double salary);
15     double getSalary() const;
16
17     double earnings() const;
18     std::string toString() const;
19 private:
20     std::string m_name{};
21     double m_salary{0.0};
22 };
```

Relationship Between Base and Derived Classes (Cont.)

- Creating and Using a SalariedEmployee Class
 - Its header specifies the class's public services:
 - ◆ a constructor (line 9),
 - ◆ member function earnings (line 17),
 - ◆ member function toString (line 18) and
 - ◆ public set and get functions that manipulate the class's data members m_name and m_salary (declared in lines 20–21).

```
7  class SalariedEmployee {  
8  public:  
9      SalariedEmployee(std::string_view name, double salary);  
10  
11     void setName(std::string_view name);  
12     std::string getName() const;  
13  
14     void setSalary(double salary);  
15     double getSalary() const;  
16  
17     double earnings() const;  
18     std::string toString() const;  
19 private:  
20     std::string m_name{};  
21     double m_salary{0.0};  
22 };
```

Relationship Between Base and Derived Classes (Cont.)

- Creating and Using a SalariedEmployee Class
 - The source code file specifies the class's implementation.

```
1 // SalariedEmployee.cpp
2 // Class SalariedEmployee member-function definitions.
3 #include <format>
4 #include <stdexcept>
5 #include "SalariedEmployee.h" // SalariedEmployee class definition
6
7 // constructor
8 SalariedEmployee::SalariedEmployee(std::string_view name, double salary)
9     : m_name{name} {
10     setSalary(salary);
11 }
12
13 // set name
14 void SalariedEmployee::setName(std::string_view name) {
15     m_name = name; // should validate
16 }
17
18 // return name
19 std::string SalariedEmployee::getName() const {return m_name;}
```

Relationship Between Base and Derived Classes (Cont.)

- SalariedEmployee Constructor

- The class's constructor (lines 8–11) uses a member-initializer list to initialize `m_name`.
 - ◆ We could validate the name, perhaps by ensuring that it's of a reasonable length.
 - ◆ The constructor calls `setSalary` to validate and initialize data member `m_salary`.

```
7 // constructor
8 SalariedEmployee::SalariedEmployee(std::string_view name, double salary)
9     : m_name{name} {
10     setSalary(salary);
11 }
12
13 // set name
14 void SalariedEmployee::setName(std::string_view name) {
15     m_name = name; // should validate
16 }
17
18 // return name
19 std::string SalariedEmployee::getName() const {return m_name;}
```

Relationship Between Base and Derived Classes (Cont.)

- Creating and Using a SalariedEmployee Class
 - Member function setSalary's implementation (lines 22–28) validates its argument before modifying the data member m_salary.

```
21 // set salary
22 void SalariedEmployee::setSalary(double salary) {
23     if (salary < 0.0) {
24         throw std::invalid_argument("Salary must be >= 0.0");
25     }
26
27     m_salary = salary;
28 }
```

Relationship Between Base and Derived Classes (Cont.)

- SalariedEmployee Member Functions earnings and toString
 - Function earnings (line 34) calls getSalary and returns the result.
 - Function toString (lines 37–40) returns a string containing a SalariedEmployee object's information.

```
30 // return salary
31 double SalariedEmployee::getSalary() const {return m_salary;}
32
33 // calculate earnings
34 double SalariedEmployee::earnings() const {return getSalary();}
35
36 // return string representation of SalariedEmployee object
37 std::string SalariedEmployee::toString() const {
38     return std::format("name: {}\nsalary: ${:.2f}\n", getName(),
39         getSalary());
40 }
```

Relationship Between Base and Derived Classes (Cont.)

- Testing Class SalariedEmployee
 - The main file tests the class.

```
1 // main.cpp
2 // SalariedEmployee class test program.
3 #include <format>
4 #include <iostream>
5 #include "SalariedEmployee.h" // SalariedEmployee class definition
6
7 int main() {
8     // instantiate a SalariedEmployee object
9     SalariedEmployee employee("Sierra Dembo", 300.0);
10
11     // get SalariedEmployee data
12     std::cout << "Employee information obtained by get functions:\n"
13         << std::format("name: {}\nsalary: ${:.2f}\n", employee.getName(),
14             employee.getSalary());
15
16     employee.setSalary(500.0); // change salary
17     std::cout << "\nUpdated employee information from function toString:\n"
18         << employee.toString();
19
20     // display only the employee's earnings
21     std::cout << std::format("\nearnings: ${:.2f}\n", employee.earnings());
22 }
```

Employee information obtained by get functions:

name: Sierra Dembo

salary: \$300.00

Updated employee information from function toString:

name: Sierra Dembo

salary: \$500.00

earnings: \$500.00

Relationship Between Base and Derived Classes (Cont.)

- Testing Class SalariedEmployee

- Line 9 creates the SalariedEmployee object employee.
- Lines 12–14 demonstrate the employee's get functions.
- Line 16 uses setSalary to change the employee's m_salary value.

```
7  int main() {  
8      // instantiate a SalariedEmployee object  
9      SalariedEmployee employee("Sierra Dembo", 300.0);  
10  
11     // get SalariedEmployee data  
12     std::cout << "Employee information obtained by get functions:\n"  
13         << std::format("name: {}\nsalary: ${:.2f}\n", employee.getName(),  
14             employee.getSalary());  
15  
16     employee.setSalary(500.0); // change salary  
17     std::cout << "\nUpdated employee information from function toString:\n"  
18         << employee.toString();  
19  
20     // display only the employee's earnings  
21     std::cout << std::format("\nearnings: ${:.2f}\n", employee.earnings());  
22 }
```

Relationship Between Base and Derived Classes (Cont.)

- Testing Class SalariedEmployee

- Then, lines 17–18 call employee's toString member function to get and output the employee's updated information.
- Finally, line 21 displays the employee's earnings using the updated m_salary value.

```
7  int main() {  
8      // instantiate a SalariedEmployee object  
9      SalariedEmployee employee("Sierra Dembo", 300.0);  
10  
11     // get SalariedEmployee data  
12     std::cout << "Employee information obtained by get functions:\n"  
13         << std::format("name: {}\nsalary: ${:.2f}\n", employee.getName(),  
14             employee.getSalary());  
15  
16     employee.setSalary(500.0); // change salary  
17     std::cout << "\nUpdated employee information from function toString:\n"  
18         << employee.toString();  
19  
20     // display only the employee's earnings  
21     std::cout << std::format("\nearnings: ${:.2f}\n", employee.earnings());  
22 }
```

Relationship Between Base and Derived Classes (Cont.)

- Creating a SalariedEmployee—SalariedCommissionEmployee Inheritance Hierarchy
 - Now, let's create a SalariedCommissionEmployee class that inherits from SalariedEmployee.

```
1 // SalariedCommissionEmployee.h
2 // SalariedCommissionEmployee class derived from class SalariedEmployee.
3 #pragma once
4 #include <string>
5 #include <string_view>
6 #include "SalariedEmployee.h"
7
8 class SalariedCommissionEmployee : public SalariedEmployee {
9 public:
10     SalariedCommissionEmployee(std::string_view name, double salary,
11                               double grossSales, double commissionRate);
12
13     void setGrossSales(double grossSales);
14     double getGrossSales() const;
15
16     void setCommissionRate(double commissionRate);
17     double getCommissionRate() const;
18
19     double earnings() const;
20     std::string toString() const;
21 private:
22     double m_grossSales{0.0};
23     double m_commissionRate{0.0};
24 };;
```

Relationship Between Base and Derived Classes (Cont.)

- Creating a SalariedEmployee—SalariedCommissionEmployee Inheritance Hierarchy
 - A SalariedCommissionEmployee object is a SalariedEmployee—public inheritance passes on SalariedEmployee’s capabilities
 - A SalariedCommissionEmployee also has m_grossSales and m_commissionRate data members (lines 22–23), which we’ll multiply to calculate the commission earned.

```
21 private:  
22     double m_grossSales{0.0};  
23     double m_commissionRate{0.0};  
24 
```

Relationship Between Base and Derived Classes (Cont.)

- Creating a SalariedEmployee—
SalariedCommissionEmployee Inheritance Hierarchy
 - Lines 13–17 declare public set and get functions that manipulate the class's m_grossSales and m_commissionRate data members.

```
13 void setGrossSales(double grossSales);  
14 double getGrossSales() const;  
15  
16 void setCommissionRate(double commissionRate);  
17 double getCommissionRate() const;  
18  
19 double earnings() const;  
20 std::string toString() const;
```

Relationship Between Base and Derived Classes (Cont.)

- Inheritance

- The colon (:) in line 8 indicates inheritance, and public to its right specifies the kind of inheritance.
- In public inheritance, public base-class members remain public in the derived class and protected ones remain protected.
- The class SalariedCommissionEmployee inherits all of SalariedEmployee's members, **except its constructor(s) and destructor.**
 - ◆ Constructors and destructors are specific to the class that defines them, so derived classes have their own.

Relationship Between Base and Derived Classes (Cont.)

- SalariedCommissionEmployee Member Functions
 - Class SalariedCommissionEmployee's public services include
 - ◆ its own constructor (lines 10–11),
 - ◆ the member functions setGrossSales, getGrossSales, setCommissionRate, getCommissionRate, earnings and toString (lines 13–20), and
 - ◆ the public member functions inherited from class SalariedEmployee.

```
8  class SalariedCommissionEmployee : public SalariedEmployee {  
9  public:  
10     SalariedCommissionEmployee(std::string_view name, double salary,  
11         double grossSales, double commissionRate);  
12  
13     void setGrossSales(double grossSales);  
14     double getGrossSales() const;  
15  
16     void setCommissionRate(double commissionRate);  
17     double getCommissionRate() const;  
18  
19     double earnings() const;  
20     std::string toString() const;
```

Relationship Between Base and Derived Classes (Cont.)

- SalariedCommissionEmployee Member Functions

- Although SalariedCommissionEmployee's source code does not contain these inherited members, they're nevertheless a part of the class.
- A SalariedCommissionEmployee object also contains SalariedEmployee's private members, but they're not directly accessible within the derived class.
 - ◆ You can access them only via the inherited SalariedEmployee public (or protected) member functions.

```
8  class SalariedCommissionEmployee : public SalariedEmployee {  
9  public:  
10     SalariedCommissionEmployee(std::string_view name, double salary,  
11         double grossSales, double commissionRate);  
12  
13     void setGrossSales(double grossSales);  
14     double getGrossSales() const;  
15  
16     void setCommissionRate(double commissionRate);  
17     double getCommissionRate() const;  
18  
19     double earnings() const;  
20     std::string toString() const;
```

Relationship Between Base and Derived Classes (Cont.)

- SalariedCommissionEmployee's Implementation

- Each derived-class constructor must call a base-class constructor.
 - ◆ The SalariedCommissionEmployee constructor (lines 8–15) does this explicitly via a base-class initializer (line 11)—a member initializer that invokes the base-class constructor by name with the appropriate arguments.
 - ◆ In this case, we initialize the inherited data members with the arguments name and salary.

```
1 // SalariedCommissionEmployee.cpp
2 // Class SalariedCommissionEmployee member-function definitions.
3 #include <format>
4 #include <stdexcept>
5 #include "SalariedCommissionEmployee.h"
6
7 // constructor
8 SalariedCommissionEmployee::SalariedCommissionEmployee(
9     std::string_view name, double salary, double grossSales,
10    double commissionRate)
11     : SalariedEmployee{name, salary} { // call base-class constructor
12
13     setGrossSales(grossSales); // validate & store gross sales
14     setCommissionRate(commissionRate); // validate & store commission rate
15 }
```

Relationship Between Base and Derived Classes (Cont.)

- SalariedCommissionEmployee's Implementation

- ◆ Functions setGrossSales (lines 18–24) and setCommissionRate (lines 32–41) validate their arguments before modifying the data members m_grossSales and m_commissionRate.

```
17 // set gross sales amount
18 void SalariedCommissionEmployee::setGrossSales(double grossSales) {
19     if (grossSales < 0.0) {
20         throw std::invalid_argument("Gross sales must be >= 0.0");
21     }
22
23     m_grossSales = grossSales;
24 }

31 // return commission rate
32 void SalariedCommissionEmployee::setCommissionRate(
33     double commissionRate) {
34
35     if (commissionRate <= 0.0 || commissionRate >= 1.0) {
36         throw std::invalid_argument(
37             "Commission rate must be > 0.0 and < 1.0");
38     }
39
40     m_commissionRate = commissionRate;
41 }
```

Relationship Between Base and Derived Classes (Cont.)

- SalariedCommissionEmployee Member Function earnings
 - SalariedCommissionEmployee's earnings function (lines 49–52) redefines earnings from class SalariedEmployee to calculate a SalariedCommissionEmployee's earnings.
 - ◆ Line 50 in SalariedCommissionEmployee's version uses the expression `SalariedEmployee::earnings()` to get the portion of the earnings based on salary then adds that value to the commission to calculate the total earnings.

```
48 // calculate earnings--uses SalariedEmployee::earnings()
49 double SalariedCommissionEmployee::earnings() const {
50     return SalariedEmployee::earnings() +
51         getGrossSales() * getCommissionRate();
52 }
```

Relationship Between Base and Derived Classes (Cont.)

- SalariedCommissionEmployee Member Function earnings
 - To call a redefined base-class member function from the derived class, **place the base-class name and the scope resolution operator (::) before the base-class member-function name.**
 - ◆ SalariedEmployee:: is required here to avoid infinite recursion.
 - ◆ Also, we avoid duplicating code by calling SalariedEmployee's earnings function from SalariedCommissionEmployee's earnings function.

```
48 // calculate earnings--uses SalariedEmployee::earnings()
49 double SalariedCommissionEmployee::earnings() const {
50     return SalariedEmployee::earnings() +
51         getGrossSales() * getCommissionRate();
52 }
```

Relationship Between Base and Derived Classes (Cont.)

- SalariedCommissionEmployee Member Function toString
 - Similarly, SalariedCommissionEmployee's toString function (lines 55–59) redefines SalariedEmployee's toString function.
 - The new version returns a string containing
 - ◆ the result of calling SalariedEmployee::toString() (line 58) and
 - ◆ the SalariedCommissionEmployee's gross sales and commission rate.

```
54 // returns string representation of SalariedCommissionEmployee object
55 std::string SalariedCommissionEmployee::toString() const {
56     return std::format(
57         "{}gross sales: ${:.2f}\ncommission rate: {:.2f}\n",
58         SalariedEmployee::toString(), getGrossSales(), getCommissionRate());
59 }
```

Relationship Between Base and Derived Classes (Cont.)

- Testing Class SalariedCommissionEmployee

```
1 // main.cpp
2 // SalariedCommissionEmployee class test program.
3 #include <format>
4 #include <iostream>
5 #include "SalariedCommissionEmployee.h"
6
7 int main() {
8     // instantiate SalariedCommissionEmployee object
9     SalariedCommissionEmployee employee{"Ivano Lal", 300.0, 5000.0, .04};
10
11     // get SalariedCommissionEmployee data
12     std::cout << "Employee information obtained by get functions:\n"
13     << std::format("{}: {}\n{}: {:.2f}\n{}: {:.2f}\n{}: {:.2f}\n",
14     "name", employee.getName(), "salary", employee.getSalary(),
15     "gross sales", employee.getGrossSales(),
16     "commission", employee.getCommissionRate());
17
18     employee.setGrossSales(8000.0); // change gross sales
19     employee.setCommissionRate(0.1); // change commission rate
20     std::cout << "\nUpdated employee information from function toString:\n"
21     << employee.toString();
22
23     // display the employee's earnings
24     std::cout << std::format("\nearnings: ${:.2f}\n", employee.earnings());
25 }
```

Employee information obtained by get functions:

name: Ivano Lalsalary: \$300.00

gross sales: \$5000.00

commission: 0.04

Updated employee information from function toString:

name: Ivano Lal

salary: \$300.00

gross sales: \$8000.00

commission rate: 0.10

earnings: \$1100.00

Relationship Between Base and Derived Classes (Cont.)

- Testing Class SalariedCommissionEmployee
 - The main function creates the SalariedCommissionEmployee object employee (line 9).
 - Lines 12–16 output the employee's data by calling its get functions to retrieve the object's data member values.

```
1 // main.cpp
2 // SalariedCommissionEmployee class test program.
3 #include <format>
4 #include <iostream>
5 #include "SalariedCommissionEmployee.h"
6
7 int main() {
8     // instantiate SalariedCommissionEmployee object
9     SalariedCommissionEmployee employee{"Ivano Lal", 300.0, 5000.0, .04};
10
11     // get SalariedCommissionEmployee data
12     std::cout << "Employee information obtained by get functions:\n"
13     << std::format("{}: {}\\n{}: {:.2f}\\n{}: {:.2f}\\n{}: {:.2f}\\n",
14     "name", employee.getName(), "salary", employee.getSalary(),
15     "gross sales", employee.getGrossSales(),
16     "commission", employee.getCommissionRate());
```

Employee information obtained by get functions:
name: Ivano Lal
salary: \$300.00
gross sales: \$5000.00
commission: 0.04

Relationship Between Base and Derived Classes (Cont.)

- Testing Class SalariedCommissionEmployee

- Lines 18–19 use setGrossSales and setCommissionRate to change the employee's m_grossSales and m_commissionRate values, respectively.
- Then, lines 20–21 call employee's toString member function to show the employee's updated information.
- Finally, line 24 displays the employee's updated earnings.

```
18     employee.setGrossSales(8000.0); // change gross sales
19     employee.setCommissionRate(0.1); // change commission rate
20     std::cout << "\nUpdated employee information from function toString:\n"
21               << employee.toString();
22
23     // display the employee's earnings
24     std::cout << std::format("\nearnings: ${:.2f}\n", employee.earnings());
25 }
```

```
Updated employee information from function toString:
name: Ivano Lal
salary: $300.00
gross sales: $8000.00
commission rate: 0.10

earnings: $1100.00
```

Relationship Between Base and Derived Classes (Cont.)

- A Derived-Class Constructor Must Call Its Base Class's Constructor
 - The compiler would issue an error if SalariedCommissionEmployee's constructor did not explicitly invoke SalariedEmployee's constructor.
 - In this case, the compiler would attempt to call SalariedEmployee's default constructor, which does not exist because the base class explicitly defined a constructor.
 - **If the base class provides a default constructor, the derived-class constructor can call the base-class constructor implicitly.**

Relationship Between Base and Derived Classes (Cont.)

- Notes on Constructors in Derived Classes
 - A derived-class constructor must call its base class's constructor with any required arguments; otherwise, a compilation error occurs.
 - ◆ This ensures that inherited private base-class members, which the derived class cannot access directly, get initialized.
 - ◆ The derived class's data-member initializers are typically placed after the base-class initializer(s) in the member-initializer list.

Relationship Between Base and Derived Classes (Cont.)

- Base-Class private Members Are Not Directly Accessible in a Derived Class
 - C++ rigidly enforces restrictions on accessing private data members.
 - ◆ Even a derived class, which is intimately related to its base class, **CANNOT directly access its base class's private data.**
 - ◆ For example, class SalariedEmployee's private m_salary data member, though part of each SalariedCommissionEmployee object, cannot be accessed directly by class SalariedCommissionEmployee's member functions.
 - ◆ If they try, the compiler produces an error message, such as the following from GNU g++:
'double SalariedEmployee::m_salary' is private within this context
 - However, as you saw the example, SalariedCommissionEmployee's member functions **can access the public members inherited** from SalariedEmployee.

Relationship Between Base and Derived Classes (Cont.)

- Including the Base-Class Header in the Derived-Class Header
 - Notice that we `#include` the base class's header in the derived class's header (line 6). This is necessary for several reasons:
 - ◆ For the derived class to inherit from the base class in line 8, the compiler requires the base-class definition from `SalariedEmployee.h`.
 - ◆ The compiler determines an object's size from its class's definition,
 - so it must know the class definition to reserve the proper amount of memory for a new object.
 - A derived-class object's size depends on the data members declared explicitly in its class definition and the data members inherited from its direct and indirect base classes.
 - Including the base class's definition **allows the compiler to determine the complete memory requirements for all the data members** that contribute to a derived-class object's total size.
 - ◆ The base-class definition also enables the compiler to **determine whether the derived class uses the base class's inherited members properly**.
 - For example, the compiler uses the base class's function prototypes to validate derived-class function calls to inherited base-class functions.

Relationship Between Base and Derived Classes.

- Eliminating Repeated Code via Implementation Inheritance
 - With implementation inheritance, the base class declares the common data members and member functions of all the classes in the hierarchy.
 - ◆ When changes are required for these common features, you make the changes only in the base class.
 - ◆ The derived classes then inherit the changes and must be recompiled.
 - Without inheritance, changes would need to be made to all the source-code files that contain a copy of the code in question.

Constructors and Destructors in Derived Classes

- Order of Constructor Calls
 - Instantiating a derived-class object begins a **chain of constructor calls**.
 - ◆ Before performing its own tasks, a derived-class constructor **invokes its direct base class's constructor** either **explicitly via a base-class member initializer** or **implicitly by calling the base class's default constructor**.
 - ◆ This process continues for each level of the hierarchy until the constructor for the base class at the top of the hierarchy is called.
 - That constructor finishes executing first, and the most-derived-class constructor's body finishes executing last.

Constructors and Destructors in Derived Classes

- Order of Constructor Calls

- Each base-class constructor initializes the base-class data members that its derived classes inherit.
- In the SalariedEmployee/SalariedCommissionEmployee hierarchy we've been studying, when we create a SalariedCommissionEmployee object:
 - ◆ Its constructor immediately calls the SalariedEmployee constructor.
 - ◆ SalariedEmployee is this hierarchy's base class, so the SalariedEmployee constructor executes, initializing the SalariedCommissionEmployee object's inherited SalariedEmployee m_name and m_salary data members.
 - ◆ SalariedEmployee's constructor returns control to SalariedCommissionEmployee's constructor, which initializes m_grossSales and m_commissionRate.

Constructors and Destructors in Derived Classes (Cont.)

- Order of Destructor Calls

- When a derived-class object is destroyed, the program calls that object's destructor.
- This begins a **chain of destructor calls**.
 - ◆ The destructors execute in the reverse order of the constructors.
 - ◆ When a derived-class object's destructor is called, it performs its task, then invokes the destructor of the next class up the hierarchy.
 - ◆ This repeats until the destructor of the base class at the hierarchy's top is called.

Constructors and Destructors in Derived Classes.

- Constructors and Destructors for Composed Objects
 - Suppose we create a derived-class object where both the base class and the derived class are composed of objects of other classes. When a derived-class object is created
 - ◆ the constructors for the base class's member objects execute in the order those objects were declared,
 - ◆ then the base-class constructor body executes,
 - ◆ then the constructors for the derived class's member objects execute in the order those objects were declared in the derived class,
 - ◆ then the derived class's constructor body executes.
 - **Destructors for data members are called in the reverse order of their corresponding constructors.**

Intro to Runtime Polymorphism: Polymorphic Video Game

- Suppose we're designing a video game containing Martians, Venusians, Plutonians, SpaceShips and LaserBeams.
- Each inherits from a SpaceObject base class with the member function draw and implements this function in a manner appropriate to the derived class.

Intro to Runtime Polymorphism: Polymorphic Video Game (Cont.)

- Screen Manager

- A screen-manager program maintains a vector of SpaceObject pointers to objects of the various classes.
- The screen manager periodically sends each object the same draw message to refresh the screen.
- Each object responds in its unique way. For example,
 - ◆ a Martian might draw itself in red with the appropriate number of antennae,
 - ◆ a SpaceShip might draw itself as a silver flying saucer and
 - ◆ a LaserBeam might draw itself as a bright red beam across the screen.
- **The same draw message has many forms of results—hence the term polymorphism.**

Intro to Runtime Polymorphism: Polymorphic Video Game (Cont.)

- Adding New Classes to the System

- A polymorphic screen manager facilitates **adding new classes to a system with minimal code modifications**.
 - ◆ Suppose we want to add Mercurian objects to our game.
 - ◆ We build a class Mercurian that inherits from SpaceObject and defines draw, then add the object's address to the vector of SpaceObject pointers.
 - ◆ The screen-manager code invokes the draw member function the same way for every object the vector points to, regardless of the object's type.
 - ◆ So, the new Mercurian objects just “plug right in.”

Intro to Runtime Polymorphism: Polymorphic Video Game (Cont.)

- Adding New Classes to the System
 - Runtime polymorphism enables you to deal in generalities and let the execution-time environment concern itself with the specifics.
 - You can direct objects to behave appropriately without knowing their types, as long as they belong to the same class hierarchy and are accessed via a base-class pointer or reference.

Intro to Runtime Polymorphism: Polymorphic Video Game.

- Adding New Classes to the System
 - Runtime polymorphism promotes **extensibility**.
 - ◆ Software that invokes polymorphic behavior is written independently of the specific object types to which messages are sent.
 - ◆ Thus, new object types that can respond to existing messages can be plugged into such a system without modifying the base system.
 - ◆ **Only code that instantiates new objects must be modified to accommodate new types.**

Relationships Among Objects in an Inheritance Hierarchy

- Let's examine the relationships among classes in an inheritance hierarchy more closely.
- We use the previously defined class hierarchy to present examples that [aim base-class and derived-class pointers at base-class and derived-class objects](#), and show how those pointers can be used to invoke member functions that manipulate those objects:
 - First, we assign a derived-class object's address to a base-class pointer, then show that invoking a function via the base-class pointer invokes the base-class functionality in the derived-class object.
 - ◆ The handle's type determines which function is called.
 - Second, we assign a base-class object's address to a derived-class pointer to show the resulting compilation error.
 - ◆ We discuss the error message and investigate why the compiler does not allow such an assignment.

Relationships Among Objects in an Inheritance Hierarchy (Cont.)

- Third, we assign a derived-class object's address to a base-class pointer, then examine how the base-class pointer can be used to invoke only the base-class functionality.
 - ◆ Compilation errors occur when we attempt to invoke derived-class-only member functions through the base-class pointer.
- Finally, we introduce **virtual functions** to demonstrate how to get **runtime polymorphic behavior** from base-class pointers aimed at derived-class objects.
 - ◆ We then assign a derived-class object's address to a base-class pointer and use that pointer to invoke derived-class functionality.
 - ◆ This is precisely what we need to achieve runtime polymorphic behavior.

Relationships Among Objects in an Inheritance Hierarchy (Cont.)

- These examples demonstrate that with public inheritance, an object of a derived class can be treated as an object of its base class.
 - This enables various interesting manipulations.
 - For example, a program can create a vector of base-class pointers that point to objects of many derived-class types.
 - The compiler allows this because each derived-class object is an object of its base class.

Relationships Among Objects in an Inheritance Hierarchy.

- On the other hand, you cannot treat a base-class object as an object of a derived class.
 - For example, a SalariedEmployee is not a SalariedCommissionEmployee—it's missing the data members `m_grossSales` or `m_commissionRate` and does not have their corresponding set and get member functions.
 - **The is-a relationship applies only from a derived class to its direct and indirect base classes.**

Invoking Base-Class Functions from Derived-Class Objects

- We reuse the SalariedEmployee and SalariedCommissionEmployee classes.
- The example demonstrates aiming base-class and derived-class pointers at baseclass and derived-class objects. The first two are natural and straightforward:
 - We aim a base-class pointer at a base-class object and invoke base-class functionality.
 - We aim a derived-class pointer at a derived-class object and invoke derived-class functionality.
- Then, we demonstrate the is-a relationship between derived classes and base classes by aiming a base-class pointer at a derived-class object and showing that the base-class functionality is available in the derived-class object.

Invoking Base-Class Functions from Derived-Class Objects(Cont.)

```
1 // main.cpp
2 // Aiming base-class and derived-class pointers at base-class
3 // and derived-class objects, respectively.
4 #include <format>
5 #include <iostream>
6 #include "SalariedEmployee.h"
7 #include "SalariedCommissionEmployee.h"
8
9 int main() {
10     // create base-class object
11     SalariedEmployee salaried{"Sierra Dembo", 500.0};
12
13     // create derived-class object
14     SalariedCommissionEmployee salariedCommission{
15         "Ivano Lal", 300.0, 5000.0, .04};
16
17     // output objects salaried and salariedCommission
18     std::cout << std::format("{}:\n{}\n{}\n",
19         "DISPLAY BASE-CLASS AND DERIVED-CLASS OBJECTS",
20         salaried.toString(), // base-class toString
21         salariedCommission.toString()); // derived-class toString
22
23     // natural: aim base-class pointer at base-class object
24     SalariedEmployee* salariedPtr{&salaried};
25     std::cout << std::format("{}:\n{}:\n{}\n",
26         "CALLING TOSTRING WITH BASE-CLASS POINTER TO",
27         "BASE-CLASS OBJECT INVOKES BASE-CLASS FUNCTIONALITY",
28         salariedPtr->toString()); // base-class version
29 }
```

DISPLAY BASE-CLASS AND DERIVED-CLASS OBJECTS:

name: Sierra Dembo
salary: \$500.00

name: Ivano Lal
salary: \$300.00
gross sales: \$5000.00
commission rate: 0.04

CALLING TOSTRING WITH BASE-CLASS POINTER TO BASE-CLASS OBJECT INVOKES BASE-CLASS FUNCTIONALITY:

name: Sierra Dembo
salary: \$500.00

Invoking Base-Class Functions from Derived-Class Objects(Cont.)

```
30 // natural: aim derived-class pointer at derived-class object
31 SalariedCommissionEmployee* salariedCommissionPtr{&salariedCommission};
32
33 std::cout << std::format("{}\n{}:\n{}\n",
34     "CALLING TOSTRING WITH DERIVED-CLASS POINTER TO",
35     "DERIVED-CLASS OBJECT INVOKES DERIVED-CLASS FUNCTIONALITY",
36     salariedCommissionPtr->toString()); // derived-class version
37
38 // aim base-class pointer at derived-class object
39 salariedPtr = &salariedCommission;
40 std::cout << std::format("{}\n{}:\n{}\n",
41     "CALLING TOSTRING WITH BASE-CLASS POINTER TO DERIVED-CLASS",
42     "OBJECT INVOKES BASE-CLASS FUNCTIONALITY",
43     salariedPtr->toString()); // base class version
44 }
```

CALLING TOSTRING WITH DERIVED-CLASS POINTER TO DERIVED-CLASS OBJECT INVOKES DERIVED-CLASS FUNCTIONALITY:

name: Ivano Lal
salary: \$300.00
gross sales: \$5000.00
commission rate: 0.04

CALLING TOSTRING WITH BASE-CLASS POINTER TO DERIVED-CLASS OBJECT INVOKES BASE-CLASS FUNCTIONALITY:

name: Ivano Lal
salary: \$300.00

Invoking Base-Class Functions from Derived-Class Objects(Cont.)

- Recall that
 - a SalariedCommissionEmployee object is a SalariedEmployee who earns a commission based on gross sales.
 - SalariedCommissionEmployee's earnings member function redefines SalariedEmployee's to include the commission.
 - SalariedCommissionEmployee's toString member function redefines SalariedEmployee's version to return the same information and the employee's commission.

Invoking Base-Class Functions from Derived-Class Objects(Cont.)

- Creating Objects and Displaying Their Contents
 - Line 11 of main creates a SalariedEmployee object and lines 14–15 create a SalariedCommissionEmployee object.
 - Lines 20 and 21 use each object's name to invoke its toString member function.

```
1 // main.cpp
2 // Aiming base-class and derived-class pointers at base-class
3 // and derived-class objects, respectively.
4 #include <format>
5 #include <iostream>
6 #include "SalariedEmployee.h"
7 #include "SalariedCommissionEmployee.h"
8
9 int main() {
10     // create base-class object
11     SalariedEmployee salaried{"Sierra Dembo", 500.0};
12
13     // create derived-class object
14     SalariedCommissionEmployee salariedCommission{
15         "Ivano Lal", 300.0, 5000.0, .04};
16
17     // output objects salaried and salariedCommission
18     std::cout << std::format("{}:\n{}\n{}\n",
19         "DISPLAY BASE-CLASS AND DERIVED-CLASS OBJECTS",
20         salaried.toString(), // base-class toString
21         salariedCommission.toString()); // derived-class toString
```

DISPLAY BASE-CLASS AND DERIVED-CLASS OBJECTS:

name: Sierra Dembo
salary: \$500.00

name: Ivano Lal
salary: \$300.00
gross sales: \$5000.00
commission rate: 0.04

Invoking Base-Class Functions from Derived-Class Objects(Cont.)

- Aiming a Base-Class Pointer at a Base-Class Object
 - Line 24 initializes the SalariedEmployee pointer salariedPtr with the base-class object salaried's address.
 - Line 28 uses the pointer to invoke the salaried object's toString member function from the base-class SalariedEmployee.

```
23 // natural: aim base-class pointer at base-class object
24 SalariedEmployee* salariedPtr{&salaried};
25 std::cout << std::format("{}\n{}\n{}\n",
26     "CALLING TOSTRING WITH BASE-CLASS POINTER TO",
27     "BASE-CLASS OBJECT INVOKES BASE-CLASS FUNCTIONALITY",
28     salariedPtr->toString()); // base-class version
29
```

CALLING TOSTRING WITH BASE-CLASS POINTER TO BASE-CLASS OBJECT INVOKES BASE-CLASS FUNCTIONALITY:
name: Sierra Dembo
salary: \$500.00

Invoking Base-Class Functions from Derived-Class Objects(Cont.)

- Aiming a Derived-Class Pointer at a Derived-Class Object
 - Line 31 initializes SalariedCommissionEmployee pointer salariedCommissionPtr with derived-class object salariedCommission's address.
 - Line 36 uses the pointer to invoke the salariedCommission object's toString member function from the derived-class SalariedCommissionEmployee.

```
30 // natural: aim derived-class pointer at derived-class object
31 SalariedCommissionEmployee* salariedCommissionPtr{&salariedCommission};
32
33 std::cout << std::format("{}\n{}:\n{}\n",
34     "CALLING TOSTRING WITH DERIVED-CLASS POINTER TO",
35     "DERIVED-CLASS OBJECT INVOKES DERIVED-CLASS FUNCTIONALITY",
36     salariedCommissionPtr->toString()); // derived-class version
```

CALLING TOSTRING WITH DERIVED-CLASS POINTER TO DERIVED-CLASS OBJECT INVOKES DERIVED-CLASS FUNCTIONALITY:

name: Ivano Lal
salary: \$300.00
gross sales: \$5000.00
commission rate: 0.04

Invoking Base-Class Functions from Derived-Class Objects.

- Aiming a Base-Class Pointer at a Derived-Class Object
 - Line 39 assigns the derived-class object `salariedCommission`'s address to `salariedPtr`—a base-class pointer.
 - ◆ This “crossover” is allowed because a derived-class object is an object of its base class.
 - Line 43 uses this pointer to invoke `toString`.
 - ◆ Even though the base-class `SalariedEmployee` pointer points to a `SalariedCommissionEmployee` derived-class object, the base class's `toString` member function is invoked.
 - ◆ The gross sales and the commission rate are **NOT** displayed because they are not base-class members.

```
38 // aim base-class pointer at derived-class object
39 salariedPtr = &salariedCommission;
40 std::cout << std::format("{}\n{}:\n{}\n",
41     "CALLING TOSTRING WITH BASE-CLASS POINTER TO DERIVED-CLASS",
42     "OBJECT INVOKES BASE-CLASS FUNCTIONALITY",
43     salariedPtr->toString()); // base class version
44 }
```

Invoking Base-Class Functions from Derived-Class Objects.

- Aiming a Base-Class Pointer at a Derived-Class Object
 - This program's output shows that the invoked function **depends on the pointer type** (or reference type) used to invoke the function, **not the object type** for which the member function is called.
 - ◆ Soon later we'll see that **virtual functions** make it possible to invoke the object type's functionality—an important aspect of implementing runtime polymorphic behavior.

```
38 // aim base-class pointer at derived-class object
39 salariedPtr = &salariedCommission;
40 std::cout << std::format("{}\n{}\n{}\n",
41     "CALLING TOSTRING WITH BASE-CLASS POINTER TO DERIVED-CLASS",
42     "OBJECT INVOKES BASE-CLASS FUNCTIONALITY",
43     salariedPtr->toString()); // base class version
44 }
```

```
CALLING TOSTRING WITH BASE-CLASS POINTER TO DERIVED-
CLASSOBJECT INVOKES BASE-CLASS FUNCTIONALITY:
name: Ivano Lal
salary: $300.00
```

Aiming Derived-Class Pointers at Base-Class Objects

- Now, let's try to aim a derived-class pointer at a base-class object.
 - Line 7 creates a SalariedEmployee object.
 - Line 11 attempts to initialize a SalariedCommissionEmployee pointer with the base-class salaried object's address.
 - The compiler generates an error because a SalariedEmployee is not a SalariedCommissionEmployee.

```
1 // main.cpp
2 // Aiming a derived-class pointer at a base-class object.
3 #include "SalariedEmployee.h"
4 #include "SalariedCommissionEmployee.h"
5
6 int main() {
7     SalariedEmployee salaried{"Sierra Dembo", 500.0};
8
9     // aim derived-class pointer at base-class object
10    // Error: a SalariedEmployee is not a SalariedCommissionEmployee
11    SalariedCommissionEmployee* salariedCommissionPtr{&salaried};
12 }
```

Aiming Derived-Class Pointers at Base-Class Objects.

- Consider the consequences if the compiler were to allow this assignment.
 - Through a SalariedCommissionEmployee pointer, we can invoke any of that class's member functions, including setGrossSales and setCommissionRate, for the object to which the pointer points—the base-class object salaried.
 - However, a SalariedEmployee object has neither setGrossSales and setCommissionRate member functions nor data members m_grossSales and m_commissionRate to set.
 - If this were allowed, it could lead to problems because member functions setGrossSales and setCommissionRate would assume data members m_grossSales and m_commissionRate exist at their “usual locations” in a SalariedCommissionEmployee object.
 - A SalariedEmployee object's memory does not have these data members, so setGrossSales and setCommissionRate might overwrite other data in memory.

Derived-Class Member-Function Calls via Base-Class Pointers

- Derived-Class Member-Function Calls via Base-Class Pointers

```
1 // main.cpp
2 // Attempting to call derived-class-only functions
3 // via a base-class pointer.
4 #include <string>
5 #include "SalariedEmployee.h"
6 #include "SalariedCommissionEmployee.h"
7
8 int main() {
9     SalariedCommissionEmployee salariedCommission{
10         "Ivano Lal", 300.0, 5000.0, .04};
11
12     // aim base-class pointer at derived-class object (allowed)
13     SalariedEmployee* salariedPtr{&salariedCommission};
14
15     // invoke base-class member functions on derived-class
16     // object through base-class pointer (allowed)
17     std::string name{salariedPtr->getName()};
18     double salary{salariedPtr->getSalary()};
19
20     // attempt to invoke derived-class-only member functions
21     // on derived-class object through base-class pointer (disallowed)
22     double grossSales{salariedPtr->getGrossSales()};
23     double commissionRate{salariedPtr->getCommissionRate()};
24     salariedPtr->setGrossSales(8000.0);
25 }
```

Derived-Class Member-Function Calls via Base-Class Pointers

- The compiler allows us to invoke only base-class member functions via a base-class pointer.
- So, a compilation error occurs if a base-class pointer is aimed at a derived-class object and an attempt is made to access a derived-class-only member function.

Derived-Class Member-Function Calls via Base-Class Pointers

- Lines 9–10 create a SalariedCommissionEmployee object.
- Line 13 initializes the base-class pointer salariedPtr with the derived-class object salariedCommission's address.
 - ◆ Again, this is allowed because a SalariedCommissionEmployee is a SalariedEmployee.
- The compiler allows us to invoke only base-class member functions via a base-class pointer.
 - ◆ Lines 17–18 invoke base-class member functions via the base-class pointer.
 - These calls are allowed because SalariedCommissionEmployee inherits these functions.

```
8 int main() {
9     SalariedCommissionEmployee salariedCommission{
10         "Ivano Lal", 300.0, 5000.0, .04};
11
12     // aim base-class pointer at derived-class object (allowed)
13     SalariedEmployee* salariedPtr{&salariedCommission};
14
15     // invoke base-class member functions on derived-class
16     // object through base-class pointer (allowed)
17     std::string name{salariedPtr->getName()};
18     double salarv{salariedPtr->getSalarv()};
```

Derived-Class Member-Function Calls via Base-Class Pointers.

- We know that `salariedPtr` is aimed at a `SalariedCommissionEmployee` object, so lines 22–24 try to invoke `SalariedCommissionEmployee`-only member functions `getGrossSales`, `getCommissionRate` and `setGrossSales`.
 - ◆ The compiler generates errors because these functions are not base-class `SalariedEmployee` member functions.
 - ◆ Through `salariedPtr`, we can invoke only member functions of the handle's class type—in this case, those of base-class `SalariedEmployee`.

```
20 // attempt to invoke derived-class-only member functions
21 // on derived-class object through base-class pointer (disallowed)
22 double grossSales{salariedPtr->getGrossSales()};
23 double commissionRate{salariedPtr->getCommissionRate()};
24 salariedPtr->setGrossSales(8000.0);
25 }
```

Virtual Functions and Virtual Destructors

- In previous example, we aimed a base-class `SalariedEmployee` pointer at a derived-class `SalariedCommissionEmployee` object, then used it to invoke the member function `toString`.
- In that case, the `SalariedEmployee` class's `toString` was called.
- How can we invoke the derived-class `toString` function via a base-class pointer?

Virtual Functions and Virtual Destructors (Cont.)

- Why virtual Functions Are Useful
 - Suppose the shape classes Circle, Triangle, Rectangle and Square derive from base-class Shape.
 - Each class might be endowed with the ability to draw objects of that class via a member function draw, but each shape's implementation is quite different. In a program that draws many different shapes, it would be convenient to treat them all generally as base-class Shape objects.

Virtual Functions and Virtual Destructors (Cont.)

- Why virtual Functions Are Useful
 - Using a base-class Shape pointer to invoke function draw, we could draw any shape.
 - ◆ At runtime, the program would dynamically determine which derived-class draw function to use based on the type of the object to which the base-class Shape pointer points.
 - ◆ This is runtime polymorphic behavior. With **virtual functions**, **the type of the object pointed to (or referenced)**—**not the type of the pointer (or reference)**— **determines which member function to invoke.**

Virtual Functions and Virtual Destructors (Cont.)

- Declaring virtual Functions

- To enable this runtime polymorphic behavior, declare the base-class member function draw as **virtual**, then **override** it in each derived class to draw the appropriate shape.
- **An overridden function in a derived class must have the same signature as the base-class function it overrides.**
- To declare a virtual function, precede its prototype with the keyword **virtual**. For example,
virtual void draw() const;
would appear in base class Shape to indicate that draw is a virtual function that takes no arguments and returns nothing.
 - ◆ This function is declared const because a draw function should not modify the Shape object on which it's invoked.
 - ◆ Virtual functions do not have to be const and can receive arguments and return values as appropriate.
 - ◆ **Once a function is declared virtual, it's virtual in all classes derived directly or indirectly from that base class.**
 - ◆ If a derived class does not override a virtual function from its base class, the derived class inherits its base class's virtual function implementation.

Virtual Functions and Virtual Destructors (Cont.)

- Invoking a virtual Function
 - If a program invokes a virtual function through
 - ◆ a base-class pointer to a derived-class object (e.g., `shapePtr->draw()`) or
 - ◆ a base-class reference to a derived-class object (e.g., `shapeRef.draw()`),
 - the program will choose the correct derived-class function at execution time based on the object's type—not the pointer or reference type.

Virtual Functions and Virtual Destructors (Cont.)

- Invoking a virtual Function
 - Choosing the appropriate function to call **at execution time** is known as **dynamic binding** or **late binding**.
 - When a virtual function is called via a specific object's name using the dot operator (e.g., `squareObject.draw()`), an optimizing compiler can resolve at compile-time the function to call.
 - ◆ This is called **static binding**.
 - ◆ The virtual function that will be called is **the one defined for that object's class**.

Virtual Functions and Virtual Destructors (Cont.)

- virtual Functions in the SalariedEmployee Hierarchy
 - Let's see how virtual functions could enable runtime polymorphic behavior in our hierarchy.
 - We made only two modifications to each class's header to enable the behavior.
 - In class SalariedEmployee's header, we modified the earnings and toString prototypes (lines 17–18) to include the virtual keyword:
virtual double earnings() const;
virtual std::string toString() const;

Virtual Functions and Virtual Destructors (Cont.)

- Class SalariedEmployee

```
1 // SalariedEmployee.h
2 // SalariedEmployee class definition.
3 #pragma once // prevent multiple inclusions of header
4 #include <string>
5 #include <string_view>
6
7 class SalariedEmployee {
8 public:
9     SalariedEmployee(std::string_view name, double salary);
10
11     void setName(std::string_view name);
12     std::string getName() const;
13
14     void setSalary(double salary);
15     double getSalary() const;
16
17     virtual double earnings() const;
18     virtual std::string toString() const;
19 private:
20     std::string m_name{};
21     double m_salary{0.0};
22 };
```

Virtual Functions and Virtual Destructors (Cont.)

- virtual Functions in the SalariedEmployee Hierarchy
 - In class SalariedCommissionEmployee's header we modified the earnings and toString prototypes (lines 19–20) to include the **override** keyword:
double earnings() const override;
std::string toString() const override;
 - The base class's earnings and toString functions are virtual, so the derived class SalariedCommissionEmployee's versions override SalariedEmployee's versions.
 - ◆ There were no changes to SalariedEmployee's or SalariedCommissionEmployee's member-function implementations, so we reused them.

Virtual Functions and Virtual Destructors (Cont.)

- Class SalariedCommissionEmployee

```
1 // SalariedCommissionEmployee.h
2 // SalariedCommissionEmployee class derived from class SalariedEmployee.
3 #pragma once
4 #include <string>
5 #include <string_view>
6 #include "SalariedEmployee.h"
7
8 class SalariedCommissionEmployee : public SalariedEmployee {
9 public:
10     SalariedCommissionEmployee(std::string_view name, double salary,
11                                double grossSales, double commissionRate);
12
13     void setGrossSales(double grossSales);
14     double getGrossSales() const;
15
16     void setCommissionRate(double commissionRate);
17     double getCommissionRate() const;
18
19     double earnings() const override;
20     std::string toString() const override;
21 private:
22     double m_grossSales{0.0};
23     double m_commissionRate{0.0};
24 };
```

Virtual Functions and Virtual Destructors (Cont.)

- Runtime Polymorphic Behavior
 - Now, if we aim a base-class SalariedEmployee pointer at a derived-class SalariedCommissionEmployee object and use that pointer to call either earnings or toString, the derived-class object's function will be invoked polymorphically.
 - The main function demonstrates this runtime polymorphic behavior.

Virtual Functions and Virtual Destructors (Cont.)

- Runtime Polymorphic Behavior

- First, lines 10–21 create a SalariedEmployee and a SalariedCommissionEmployee, then use their object names to show their toString results.

```
1 // main.cpp
2 // Introducing polymorphism, virtual functions and dynamic binding.
3 #include <format>
4 #include <iostream>
5 #include "SalariedEmployee.h"
6 #include "SalariedCommissionEmployee.h"
7
8 int main() {
9     // create base-class object
10    SalariedEmployee salaried{"Sierra Dembo", 500.0};
11
12    // create derived-class object
13    SalariedCommissionEmployee salariedCommission{
14        "Ivano Lal", 300.0, 5000.0, .04};
15
16    // output objects using static binding
17    std::cout << std::format("{}\n{}:\n{}\n{}\n",
18        "INVOKING TOSTRING FUNCTION ON BASE-CLASS AND",
19        "DERIVED-CLASS OBJECTS WITH STATIC BINDING",
20        salaried.toString(), // static binding
21        salariedCommission.toString()); // static binding
```

INVOKING TOSTRING FUNCTION ON BASE-CLASS
AND DERIVED-CLASS OBJECTS WITH STATIC BINDING:

name: Sierra Dembo
salary: \$500.00

name: Ivano Lal
salary: \$300.00
gross sales: \$5000.00
commission rate: 0.04

Virtual Functions and Virtual Destructors (Cont.)

- Runtime Polymorphic Behavior

- Lines 27–38 show again that

- ◆ a SalariedEmployee pointer aimed at a SalariedEmployee object can be used to invoke SalariedEmployee functionality and

```
23 std::cout << "INVOKING TOSTRING FUNCTION ON BASE-CLASS AND\n"
24 << "DERIVED-CLASS OBJECTS WITH DYNAMIC BINDING\n\n";
25
26 // natural: aim base-class pointer at base-class object
27 SalariedEmployee* salariedPtr(&salaried);
28 std::cout << std::format("{}\n{}:\n{}\n",
29 "CALLING VIRTUAL FUNCTION TOSTRING WITH BASE-CLASS POINTER",
30 "TO BASE-CLASS OBJECT INVOKES BASE-CLASS FUNCTIONALITY",
31 salariedPtr->toString()); // base-class version
```

INVOKING TOSTRING FUNCTION ON BASE-CLASS AND DERIVED-CLASS OBJECTS WITH DYNAMIC BINDING

CALLING VIRTUAL FUNCTION TOSTRING WITH BASE-CLASS POINTERTO BASE-CLASS OBJECT INVOKES BASE-CLASS FUNCTIONALITY:

name: Sierra Dembo

salary: \$500.00

Virtual Functions and Virtual Destructors (Cont.)

- Runtime Polymorphic Behavior

- Lines 27–38 show again that

- ◆ a SalariedCommissionEmployee pointer aimed at a SalariedCommissionEmployee object can be used to invoke SalariedCommissionEmployee functionality.

```
33 // natural: aim derived-class pointer at derived-class object
34 SalariedCommissionEmployee* salariedCommissionPtr(&salariedCommission);
35 std::cout << std::format("{}\n{}:\n{}\n",
36     "CALLING VIRTUAL FUNCTION TOSTRING WITH DERIVED-CLASS POINTER",
37     "TO DERIVED-CLASS OBJECT INVOKES DERIVED-CLASS FUNCTIONALITY",
38     salariedCommissionPtr->toString()); // derived-class version
```

```
CALLING VIRTUAL FUNCTION TOSTRING WITH DERIVED-CLASS POINTER TO DERIVED-CLASS OBJECT INVOKES DERIVED-CLASS FUNCTIONALITY:
name: Ivano Lal
salary: $300.00
gross sales: $5000.00
commission rate: 0.04
```

Virtual Functions and Virtual Destructors (Cont.)

- Runtime Polymorphic Behavior
 - **Declaring the member function virtual and invoking it through a base-class pointer or reference causes the program to determine at execution time which function to invoke based on the object's type.**

Virtual Functions and Virtual Destructors (Cont.)

- Runtime Polymorphic Behavior

- When `salariedPtr` points to a `SalariedEmployee` object, class `SalariedEmployee`'s `toString` function is invoked (line 31).

```
26 // natural: aim base-class pointer at base-class object
27 SalariedEmployee* salariedPtr(&salaried);
28 std::cout << std::format("{}\n{}:\n{}\n",
29     "CALLING VIRTUAL FUNCTION TOSTRING WITH BASE-CLASS POINTER",
30     "TO BASE-CLASS OBJECT INVOKES BASE-CLASS FUNCTIONALITY",
31     salariedPtr->toString()); // base-class version
```

```
CALLING VIRTUAL FUNCTION TOSTRING WITH BASE-CLASS POINTERTO BASE-CLASS OBJECT INVOKES BASE-CLASS FUNCTIONALITY:
name: Sierra Dembo
salary: $500.00
```

Virtual Functions and Virtual Destructors (Cont.)

- Runtime Polymorphic Behavior

- Line 41 aims the base-class pointer `salariedPtr` at the derived-class object `salariedCommission`.
- Line 48 invokes `toString` via the base-class pointer.
- As you can see in the output, the derived-class `salariedCommission` object's `toString` is invoked.

```
40 // aim base-class pointer at derived-class object
41 salariedPtr = &salariedCommission;
42
43 // runtime polymorphism: invokes SalariedCommissionEmployee
44 // via base-class pointer to derived-class object
45 std::cout << std::format("{}\n{}\n{}\n",
46     "CALLING VIRTUAL FUNCTION TOSTRING WITH BASE-CLASS POINTER",
47     "TO DERIVED-CLASS OBJECT INVOKES DERIVED-CLASS FUNCTIONALITY",
48     salariedPtr->toString()); // derived-class version
```

```
CALLING VIRTUAL FUNCTION TOSTRING WITH BASE-CLASS POINTER TO DERIVED-CLASSOBJECT INVOKES BASE-CLASS FUNCTIONALITY:
name: Ivano Lal
salary: $300.00
gross sales: $5000.00
commission rate: 0.04
```

Virtual Functions and Virtual Destructors (Cont.)

- Runtime Polymorphic Behavior

- When `salariedPtr` points to a `SalariedCommissionEmployee` object, class `SalariedCommissionEmployee`'s `toString` function is invoked (line 48).
 - ◆ So, the same `toString` call via a base-class pointer to various objects takes on many forms (in this case, two forms).
 - ◆ This is runtime polymorphic behavior.

```
40 // aim base-class pointer at derived-class object
41 salariedPtr = &salariedCommission;
42
43 // runtime polymorphism: invokes SalariedCommissionEmployee
44 // via base-class pointer to derived-class object
45 std::cout << std::format("{}\n{}\n{}\n",
46     "CALLING VIRTUAL FUNCTION TOSTRING WITH BASE-CLASS POINTER",
47     "TO DERIVED-CLASS OBJECT INVOKES DERIVED-CLASS FUNCTIONALITY",
48     salariedPtr->toString()); // derived-class version
```

```
CALLING VIRTUAL FUNCTION TOSTRING WITH BASE-CLASS POINTER TO DERIVED-CLASSOBJECT INVOKES BASE-CLASS FUNCTIONALITY:
name: Ivano Lal
salary: $300.00
gross sales: $5000.00
commission rate: 0.04
```

Virtual Functions and Virtual Destructors (Cont.)

- Do Not Call Virtual Functions from Constructors and Destructors
 - **Calling a virtual function from a base class's constructor or destructor invokes the base-class version**, even if the base-class constructor or destructor is called while creating or destroying a derived-class object.
 - **This is NOT the behavior you expect** for virtual functions, so the C++ Core Guidelines recommend that you **do NOT call them from constructors or destructors**.

Virtual Functions and Virtual Destructors (Cont.)

- override Keyword

- Declaring SalariedCommissionEmployee's earnings and toString functions using the **override** keyword tells the compiler **to check whether the base class has a virtual member function with the same signature**.
 - ◆ If not, the compiler generates an error.
 - ◆ This ensures that you override the appropriate base-class function.
 - ◆ It also prevents you from accidentally hiding a base-class function with the same name but a different signature.
 - ◆ So, to help you prevent errors, apply override to the prototype of every derived-class function that overrides a virtual base-class function.
- The C++ Core Guidelines state that
 - ◆ **virtual specifically introduces a new virtual function** in a hierarchy and
 - ◆ **override specifically indicates that a derived-class function overrides a base-class virtual function**.
- So, you should **use only virtual OR override** in each virtual function's prototype.

Virtual Functions and Virtual Destructors (Cont.)

- virtual Destructors

- The C++ Core Guidelines recommend **including a virtual destructor in every class that contains virtual functions.**
- This helps prevent subtle errors when a derived class has a custom destructor.
- In a class that does not have a destructor, the compiler generates one for you, but **the generated one is not virtual.**
- So, in Modern C++, the virtual destructor definition for most classes is written as
virtual ~SalariedEmployee() = default;
 - ◆ This enables you to declare the destructor virtual and still have the compiler generate a default destructor for the class—via the notation **= default.**

Virtual Functions and Virtual Destructors (Cont.)

- final Member Functions and Classes
 - A virtual function that's declared final in its prototype, as in `returnType someFunction(parameters) final;` cannot be overridden in any derived class.
 - **In a multilevel class hierarchy, this guarantees that the final member-function definition will be used by all subsequent direct and indirect derived classes.**
 - You can declare a class final to prevent it from being used as a base class, as in

```
class MyClass final {  
    // class body  
};  
or  
class DerivedClass : public BaseClass final {  
    // class body  
};
```

Virtual Functions and Virtual Destructors.

- final Member Functions and Classes

- Attempting to override a final member function or inherit from a final base class results in a compilation error.
- A benefit of declaring a virtual function final is that once the compiler knows a virtual function cannot be overridden, it can perform various optimizations.
 - ◆ For instance, the compiler might be able to determine at compile time the correct function to call.
 - ◆ This optimization is called [devirtualization](#).
- There are cases in which the compiler can devirtualize virtual function calls even if the virtual functions are not declared final.
 - ◆ For example, sometimes, the compiler can recognize the type of object that will be used at runtime.
 - ◆ In this case, **a call to a non-final virtual function can be bound at compile-time.**

Abstract Classes and Pure virtual Functions

- There are cases in which it's useful to define classes from which you never intend to instantiate any objects.
- Such an **abstract class** defines a common public interface for the classes derived from it.
- Because abstract classes are used as base classes in inheritance hierarchies, we refer to them as abstract base classes.
 - Such classes cannot be used to create objects because, as we'll see, abstract classes are missing pieces.
 - Derived classes must define the “missing pieces” before derived-class objects can be instantiated.

Abstract Classes and Pure virtual Functions

- Classes that can be used to instantiate objects are called **concrete classes**.
- Such classes define or inherit implementations of every member function they or their base classes declare.
- A good example of an abstract class is Shape's class hierarchy.
 - We then have an abstract base class, TwoDimensionalShape, with derived concrete classes Circle, Square and Triangle.
 - We also have an abstract base class, ThreeDimensionalShape, with derived concrete classes Cube, Sphere and Tetrahedron.
 - **Abstract base classes are too general to define objects.**
 - ◆ For example, if someone tells you to “draw the two-dimensional shape,” what shape would you draw?
 - **Concrete classes provide the specifics that make it possible to instantiate objects.**

Abstract Classes and Pure virtual Functions (Cont.)

- Pure virtual Functions

- A class is made abstract by declaring one or more pure virtual functions, each specified by placing “= 0” in its function prototype, as in
virtual void draw() const = 0; // pure virtual function
 - ◆ The “= 0” is a pure specifier.
- Pure virtual functions do **NOT** provide implementations.
 - ◆ Each concrete derived class must override its base class’s pure virtual functions with concrete implementations.
 - ◆ Otherwise, the derived class is also abstract.
- By contrast, a regular virtual function has an implementation, giving the derived class the option of overriding the function or simply inheriting the base class’s implementation.
- An abstract class also can have data members and concrete functions.
- **An abstract class with all pure virtual functions is sometimes called a pure abstract class or an interface.**

Abstract Classes and Pure virtual Functions (Cont.)

- Pure virtual Functions

- **A pure virtual function is used when the base class does not know how to implement a function, but all concrete derived classes should implement it.**
 - ◆ Returning to our earlier SpaceObjects example, it does not make sense for the base class SpaceObject to implement a draw function.
 - ◆ There's no way to draw a generic space object without knowing which specific type of space object is being drawn.
- **Although we cannot instantiate objects of an abstract base class, we can declare pointers and references of the abstract-base-class type.**
 - ◆ These can refer to objects of any concrete classes derived from the abstract base class.
 - ◆ Programs typically use such pointers and references to manipulate derived-class objects polymorphically at runtime.

Abstract Classes and Pure virtual Functions (Cont.)

- Device Drivers: Polymorphism in Operating Systems
 - Polymorphism is particularly effective for implementing layered software systems.
 - For example, in operating systems, each type of physical input/output device could operate quite differently from the others.
 - Even so, commands to read or write data from and to devices may have a certain uniformity.
 - The write message sent to a device-driver object needs to be interpreted specifically in the context of that device driver and how that device driver manipulates devices of a given type.
 - However, the write call itself is no different from the write to any other device in the system—place some bytes from memory onto that device.

Abstract Classes and Pure virtual Functions (Cont.)

- Device Drivers: Polymorphism in Operating Systems
 - Consider an object-oriented operating system that uses an abstract base class to provide an interface appropriate for all device drivers.
 - Through inheritance from that abstract base class, derived classes are formed that all operate similarly.
 - The public functions offered by the device drivers are pure virtual functions in the abstract base class.
 - The implementations of these pure virtual functions are provided in the derived classes that correspond to the specific types of device drivers.

Abstract Classes and Pure virtual Functions.

- Device Drivers: Polymorphism in Operating Systems
 - This architecture also allows new devices to be added to a system easily.
 - The user can just plug in the device and install its device driver.
 - The operating system “talks” to this new device through its device driver, which has the same public member functions as all other device drivers—those defined in the device-driver abstract base class.

Case Study: Payroll System Using Runtime Polymorphism

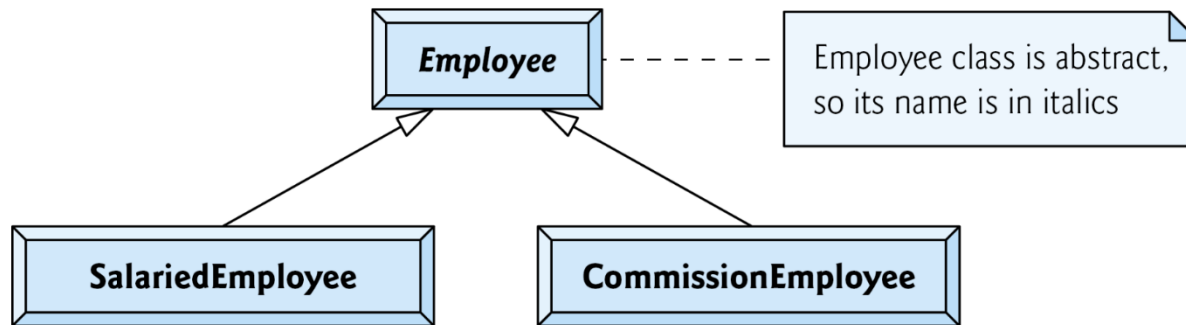
- Let's use an abstract class and runtime polymorphism to perform payroll calculations for two employee types. We create an Employee hierarchy to solve the following problem:
- A company pays its employees weekly. The employees are of two types:
 - Salaried employees are paid a fixed salary regardless of their hours worked.
 - Commission employees are paid a percentage of their sales.
- The company wants to implement a C++ program that performs its payroll calculations polymorphically.

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Many hierarchies start with an abstract base class followed by a row of derived classes that are final.
 - We use the abstract class `Employee` to represent the general concept of an employee and define the “interface” to the hierarchy—that is, the member functions all `Employee` derived classes must have, so a program can invoke them on all `Employee` objects.
 - Also, regardless of how earnings are calculated, each employee has a name. So, we’ll define a private data member `m_name` in `Employee`.
 - `SalariedEmployee` and `CommissionEmployee` are final classes that we derive directly from `Employee`.
 - ◆ Each is a leaf node in the class hierarchy and cannot be a base class.

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- The following UML class diagram shows our runtime polymorphic payroll application's inheritance hierarchy.
- The abstract class Employee is italicized, as per the UML's convention.



Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- A derived class can inherit interface and/or implementation from a base class.
- Hierarchies designed for **interface inheritance** tend to have **concrete definitions of their functionality lower in the hierarchy**.
 - A base class declares one or more functions that should be defined by every derived class, but the individual **derived classes provide the implementations** of the function(s).
- In the next couple of examples we show the Employee class hierarchy.
 - The first three each define one of the abstract or concrete classes.
 - The last contains a test program that builds concrete-class objects and processes them with runtime polymorphism.

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Creating Abstract Base Class Employee
 - Class Employee provides functions earnings and toString, and get and set functions that manipulate Employee's m_name data member.
 - An earnings function generally applies to all Employees, but each calculation depends on the kind of Employee.
 - ◆ So we declare earnings as pure virtual in base class Employee because a default implementation does not make sense for that function.
 - ◆ There's not enough information to determine what amount earnings should return.

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Creating Abstract Base Class Employee
 - Each derived class overrides earnings with an appropriate implementation.
 - ◆ To calculate an employee's earnings, the program assigns an employee object's address to a base-class Employee pointer, then invokes the object's earnings function.
 - The test program contains a vector of Employee pointers.
 - ◆ Each points to an object that is an Employee—specifically, any object of a concrete derived class of Employee.
 - The program iterates through the vector and polymorphically calls each Employee's earnings function.
 - Making earnings a pure virtual function in Employee forces every derived class of Employee that wishes to be a concrete class to override earnings.

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Creating Abstract Base Class Employee
 - Employee's toString function returns a string containing the Employee's name.
 - ◆ Each class derived from Employee will override toString to return the name followed by the rest of the Employee's information.
 - Each derived class's toString could also call earnings, even though earnings is a pure virtual function in Employee.
 - ◆ Each concrete class is guaranteed to have an earnings implementation.
 - ◆ Even class Employee's toString function can call earnings.
 - ◆ When you call toString through an Employee pointer or reference at runtime, you always call it on a concrete derived-class object.

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Creating Abstract Base Class Employee
 - The following diagram shows the hierarchy's three classes down the left and functions earnings and toString across the top.
 - For each class, the diagram shows the desired return value of each function.

	earnings	toString
Employee	pure virtual	name: <i>m_name</i>
Salaried- Employee	<i>m_salary</i>	name: <i>m_name</i> salary: <i>m_salary</i>
Commission- Employee	<i>m_commissionRate * m_grossSales</i>	name: <i>m_name</i> gross sales: <i>m_grossSales</i> commission rate: <i>m_commissionRate</i>

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Creating Abstract Base Class Employee
 - Italic text represents where the values from a particular object are used in the earnings and toString functions.
 - Class Employee specifies “pure virtual” for function earnings to indicate it does not provide an implementation.
 - Each derived class overrides this function to provide an appropriate implementation.
 - We do not list base class Employee’s get and set functions because the derived classes do not override them.
 - ◆ Each function is inherited and used “as is” by the derived classes.

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Employee Class Header

- Consider class Employee's header. Its public member functions include

- ◆ a constructor that takes the name as an argument (line 9),
- ◆ a defaulted virtual destructor (line 10),
- ◆ a set function to set the name (line 12),
- ◆ a get function to return the name (line 13),
- ◆ pure virtual function earnings (line 16) and
- ◆ virtual function toString (line 17).

```
7  class Employee {  
8  public:  
9      explicit Employee(std::string_view name);  
10     virtual ~Employee() = default; // compiler generates virtual destructor  
11  
12     void setName(std::string_view name);  
13     std::string getName() const;  
14  
15     // pure virtual function makes Employee an abstract base class  
16     virtual double earnings() const = 0; // pure virtual  
17     virtual std::string toString() const; // virtual  
18 private:  
19     std::string m_name;  
20 };
```

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Employee Class Header

- Recall that we declared earnings as a pure virtual function because we must know the specific Employee type to determine the appropriate earnings calculation.
 - ◆ Each concrete derived class must provide an earnings implementation.
 - ◆ Then, using a base-class Employee pointer or reference, a program can invoke function earnings polymorphically for an object of any concrete derived class of Employee.

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Employee Class Member-Function Definitions

- In the Employee class's member-function definitions, no implementation is provided for virtual function earnings.
 - ◆ The virtual function toString implementation (lines 17–19) will be overridden in each derived class.
 - ◆ Those derived-class toString functions will call Employee's toString to get a string containing the information common to all classes in the Employee hierarchy (i.e., the name).

```
1 // Employee.cpp
2 // Abstract-base-class Employee member-function definitions.
3 // Note: No definitions are given for pure virtual functions.
4 #include <format>
5 #include "Employee.h" // Employee class definition
6
7 // constructor
8 Employee::Employee(std::string_view name) : m_name{name} {} // empty body
9
10 // set name
11 void Employee::setName(std::string_view name) {m_name = name;}
12
13 // get name
14 std::string Employee::getName() const {return m_name;}
15
16 // return string representation of an Employee
17 std::string Employee::toString() const {
18     return std::format("name: {}", getName());
19 }
```

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Creating Concrete Derived Class SalariedEmployee
 - Class SalariedEmployee derives from class Employee.

```
1 // SalariedEmployee.h
2 // SalariedEmployee class derived from Employee.
3 #pragma once
4 #include <string> // C++ standard string class
5 #include <string_view>
6 #include "Employee.h" // Employee class definition
7
8 class SalariedEmployee final : public Employee {
9 public:
10     SalariedEmployee(std::string_view name, double salary);
11     virtual ~SalariedEmployee() = default; // virtual destructor
12
13     void setSalary(double salary);
14     double getSalary() const;
15
16     // keyword override signals intent to override
17     double earnings() const override; // calculate earnings
18     std::string toString() const override; // string representation
19 private:
20     double m_salary{0.0};
21 };
```

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Creating Concrete Derived Class SalariedEmployee
 - SalariedEmployee's public member functions include
 - ◆ a constructor that takes a name and a salary as arguments (line 10),
 - ◆ a default virtual destructor (line 11),
 - ◆ a set function to assign a new non-negative value to data member m_salary (line 13) and a get function to return m_salary's value (line 14),

```
8  class SalariedEmployee final : public Employee {
9  public:
10     SalariedEmployee(std::string_view name, double salary);
11     virtual ~SalariedEmployee() = default; // virtual destructor
12
13     void setSalary(double salary);
14     double getSalary() const;
15
16     // keyword override signals intent to override
17     double earnings() const override; // calculate earnings
18     std::string toString() const override; // string representation
19 private:
20     double m_salary{0.0};
21 };
```

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Creating Concrete Derived Class SalariedEmployee
 - SalariedEmployee's public member functions include
 - ◆ an override of Employee's virtual function earnings that calculates a SalariedEmployee's earnings (line 17) and
 - ◆ an override of Employee's virtual function toString (line 18) that returns a SalariedEmployee's string representation.

```
8  class SalariedEmployee final : public Employee {
9  public:
10     SalariedEmployee(std::string_view name, double salary);
11     virtual ~SalariedEmployee() = default; // virtual destructor
12
13     void setSalary(double salary);
14     double getSalary() const;
15
16     // keyword override signals intent to override
17     double earnings() const override; // calculate earnings
18     std::string toString() const override; // string representation
19 private:
20     double m_salary{0.0};
21 };
```

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- SalariedEmployee Class Member-Function Definitions
 - The SalariedEmployee's member functions define:
 - ◆ Its constructor passes name to the Employee constructor (line 9) to initialize the inherited private data member that's not directly accessible in the derived class.

```
1 // SalariedEmployee.cpp
2 // SalariedEmployee class member-function definitions.
3 #include <format>
4 #include <stdexcept>
5 #include "SalariedEmployee.h" // SalariedEmployee class definition
6
7 // constructor
8 SalariedEmployee::SalariedEmployee(std::string_view name, double salary)
9 {
10     Employee{name} {
11         setSalary(salary);
12     }
13
14 // set salary
15 void SalariedEmployee::setSalary(double salary) {
16     if (salary < 0.0) {
17         throw std::invalid_argument("Weekly salary must be >= 0.0");
18     }
19     m_salary = salary;
20 }
```

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- SalariedEmployee Class Member-Function Definitions
 - The SalariedEmployee's member functions define:
 - ◆ Function earnings (line 27) overrides Employee's pure virtual function earnings to provide a concrete implementation that returns the weekly salary.
 - If we did not override earnings, SalariedEmployee would inherit Employee's pure virtual earnings function and be abstract.

```
23 double SalariedEmployee::getSalary() const {return m_salary;}
24
25 // calculate earnings;
26 // override pure virtual function earnings in Employee
27 double SalariedEmployee::earnings() const {return getSalary();}
28
29 // return a string representation of SalariedEmployee
30 std::string SalariedEmployee::toString() const {
31     return std::format("{}\n{}: ${:.2f}", Employee::toString(),
32         "salary", getSalary());
33 }
```

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- SalariedEmployee Class Member-Function Definitions
 - The SalariedEmployee's member functions define:
 - ◆ SalariedEmployee's toString function (lines 30–33) overrides Employee's toString.
 - If it did not, the class would inherit Employee's, which returns a string containing the employee's name.
 - SalariedEmployee's toString returns a string containing the Employee::toString() result and the SalariedEmployee's salary.
 - SalariedEmployee's header declared member functions earnings and toString as override to ensure that we correctly override them.
 - ◆ These are virtual in base class Employee, so they remain virtual throughout the class hierarchy.

```
29 // return a string representation of SalariedEmployee
30 std::string SalariedEmployee::toString() const {
31     return std::format("{}\n{: ${{:.2f}}", Employee::toString(),
32         "salary", getSalary());
33 }
```

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Creating Concrete Derived Class
CommissionEmployee
 - Class CommissionEmployee derives from Employee.

```
1 // CommissionEmployee.h
2 // CommissionEmployee class derived from Employee.
3 #pragma once
4 #include <string>
5 #include <string_view>
6 #include "Employee.h" // Employee class definition
7
8 class CommissionEmployee final : public Employee {
9 public:
10     CommissionEmployee(std::string_view name, double grossSales,
11                       double commissionRate);
12     virtual ~CommissionEmployee() = default; // virtual destructor
13
14     void setGrossSales(double grossSales);
15     double getGrossSales() const;
16
17     void setCommissionRate(double commissionRate);
18     double getCommissionRate() const;
19
20     // keyword override signals intent to override
21     double earnings() const override; // calculate earnings
22     std::string toString() const override; // string representation
23 private:
24     double m_grossSales{0.0};
25     double m_commissionRate{0.0};
26 };
```

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Creating Concrete Derived Class
CommissionEmployee

- The member-function implementations include
 - ◆ a constructor (lines 8–12) that takes a name, gross sales and commission rate, then passes the name to Employee's constructor (line 9) to initialize the inherited data member,

```
1 // CommissionEmployee.cpp
2 // CommissionEmployee class member-function definitions.
3 #include <format>
4 #include <stdexcept>
5 #include "CommissionEmployee.h" // CommissionEmployee class definition
6
7 // constructor
8 CommissionEmployee::CommissionEmployee(std::string_view name,
9 double grossSales, double commissionRate) : Employee{name} {
10     setGrossSales(grossSales);
11     setCommissionRate(commissionRate);
12 }
```

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Creating Concrete Derived Class
CommissionEmployee

- The member-function implementations include

- ◆ set functions (lines 15–21 and 27–34) to assign new values to data members m_grossSales and m_commissionRate,

```
14 // set gross sales amount
15 void CommissionEmployee::setGrossSales(double grossSales) {
16     if (grossSales < 0.0) {
17         throw std::invalid_argument("Gross sales must be >= 0.0");
18     }
19
20     m_grossSales = grossSales;
21 }

26 // set commission rate
27 void CommissionEmployee::setCommissionRate(double commissionRate) {
28     if (commissionRate <= 0.0 || commissionRate >= 1.0) {
29         throw std::invalid_argument(
30             "Commission rate must be > 0.0 and < 1.0");
31     }
32
33     m_commissionRate = commissionRate;
34 }
```

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Creating Concrete Derived Class
CommissionEmployee
 - The member-function implementations include
 - ◆ get functions (lines 24 and 37–39) to return the values of m_grossSales and m_commissionRate,

```
23 // return gross sales amount
24 double CommissionEmployee::getGrossSales() const {return m_grossSales;}

36 // return commission rate
37 double CommissionEmployee::getCommissionRate() const {
38     return m_commissionRate;
39 }
```

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Creating Concrete Derived Class
CommissionEmployee

- The member-function implementations include
 - ◆ an override of Employee's earnings function (lines 42–44) that calculates a CommissionEmployee's earnings and
 - ◆ an override of Employee's toString function (lines 47–51) to return a string containing the Employee::toString() result, gross sales and commission rate.

```
41 // calculate earnings
42 double CommissionEmployee::earnings() const {
43     return getGrossSales() * getCommissionRate();
44 }
45
46 // return string representation of CommissionEmployee object
47 std::string CommissionEmployee::toString() const {
48     return std::format("{}\n{}: ${:.2f}\n{}: {:.2f}", Employee::toString(),
49         "gross sales", getGrossSales(),
50         "commission rate", getCommissionRate());
51 }
```

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Demonstrating Runtime Polymorphic Processing
 - To test our Employee hierarchy, the main program creates an object of each concrete class—SalariedEmployee and CommissionEmployee.
 - The program manipulates these objects first via their object names, then with runtime polymorphism, using a vector of Employee base-class pointers.

```
1 // main.cpp
2 // Processing Employee derived-class objects with variable-name handles
3 // then polymorphically using base-class pointers and references
4 #include <format>
5 #include <iostream>
6 #include <vector>
7 #include "Employee.h"
8 #include "SalariedEmployee.h"
9 #include "CommissionEmployee.h"
10
11 void virtualViaPointer(const Employee* baseClassPtr); // prototype
12 void virtualViaReference(const Employee& baseClassRef); // prototype
```

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Demonstrating Runtime Polymorphic Processing
 - Lines 16–17 create objects of each concrete derived class.
 - Lines 20–23 output each employee's information and earnings.
 - **These lines use the variable names for each object, so the compiler can identify each object's type at compile time to determine which toString and earnings functions to call.**

```
14 int main() {  
15     // create derived-class objects  
16     SalariedEmployee salaried{"Pierre Simon", 800.0};  
17     CommissionEmployee commission{"Sierra Dembo", 10000, .06};  
18  
19     // output each Employee  
20     std::cout << "EMPLOYEES PROCESSED INDIVIDUALLY VIA VARIABLE NAMES\n"  
21     << std::format("{}\n{}\n{}\n{}\n{}\n{}\n",  
22         salaried.toString(), "earned $", salaried.earnings(),  
23         commission.toString(), "earned $", commission.earnings());  
}
```

EMPLOYEES PROCESSED INDIVIDUALLY VIA
VARIABLE NAMES
name: Pierre Simon
salary: \$800.00
earned \$800.00

name: Sierra Dembo
gross sales: \$10000.00
commission rate: 0.06
earned \$600.00

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Creating a vector of Employee Pointers

- Line 26 creates and initializes the vector `employees`, containing two `Employee` pointers aimed at the objects `salaried` and `commission`.

```
25 // create and initialize vector of base-class pointers
26 std::vector<Employee*> employees{&salaried, &commission};
27
```

- The compiler allows the elements to be initialized with these objects's addresses because
 - ◆ a `SalariedEmployee` is an `Employee` and
 - ◆ a `CommissionEmployee` is an `Employee`.

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Function `virtualViaPointer`

- Lines 35–37 traverse the vector `employees` and invoke `virtualViaPointer` (lines 50–53) with each element as an argument.
 - ◆ This function receives in its `baseClassPtr` parameter the address stored in a given `employees` element, then uses the pointer to invoke virtual functions `toString` and `earnings`.
 - ◆ The function does not contain any `SalariedEmployee` or `CommissionEmployee` type information—it knows only about base class `Employee`.

```
35  for (const Employee* employeePtr : employees) {  
36      virtualViaPointer(employeePtr);  
37  }  
  
48  // call Employee virtual functions toString and earnings via a  
49  // base-class pointer using dynamic binding  
50  void virtualViaPointer(const Employee* baseClassPtr) {  
51      std::cout << std::format("{}\nearned ${:.2f}\n\n",  
52          baseClassPtr->toString(), baseClassPtr->earnings());  
53  }
```

name: Pierre Simon
salary: \$800.00
earned \$800.00

name: Sierra Dembo
gross sales: \$10000.00
commission rate: 0.06
earned \$600.00

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Function virtualViaPointer

- The program repeatedly aims baseClassPtr at different concrete derived-class objects, so the compiler cannot know which concrete class's functions to call through baseClassPtr.
 - ◆ It must resolve these calls at runtime using dynamic binding.
 - ◆ At execution time, each call correctly invokes the virtual function on the object to which baseClassPtr currently points.
- The output shows that each class's appropriate functions are invoked and display the correct information.
 - ◆ Obtaining each Employee's earnings via runtime polymorphism produces the same results as lines 22 and 23.

```
48 // call Employee virtual functions toString and earnings via a
49 // base-class pointer using dynamic binding
50 void virtualViaPointer(const Employee* baseClassPtr) {
51     std::cout << std::format("{}\nearned ${:.2f}\n\n",
52                               baseClassPtr->toString(), baseClassPtr->earnings());
53 }
```

Case Study: Payroll System Using Runtime Polymorphism (Cont.)

- Function `virtualViaReference`

- Lines 43–45 traverse employees and invoke `virtualViaReference` (lines 57–60) with each element as an argument.

- ◆ This function receives in its `baseClassRef` parameter (of type `const Employee&`) a reference to the object obtained by dereferencing the pointer stored in an element of vector `employees` (line 44).

```
43  for (const Employee* employeePtr : employees) {  
44      virtualViaReference(*employeePtr); // note dereferenced pointer  
45  }  
46
```

```
55  // call Employee virtual functions toString and earnings via a  
56  // base-class reference using dynamic binding  
57  void virtualViaReference(const Employee& baseClassRef) {  
58      std::cout << std::format("{}\nearned {:.2f}\n\n",  
59          baseClassRef.toString(), baseClassRef.earnings());  
60  }
```

name: Pierre Simon
salary: \$800.00
earned \$800.00

name: Sierra Dembo
gross sales: \$10000.00
commission rate: 0.06
earned \$600.00

Case Study: Payroll System Using Runtime Polymorphism.

- Function virtualViaReference

- ◆ Each call to this function invokes virtual functions toString and earnings via baseClassRef to demonstrate that runtime polymorphic processing also occurs with base-class references.
- ◆ Each function invocation calls the virtual function on the object baseClassRef references at runtime— another example of dynamic binding.
- ◆ The output produced using base-class references is identical to that produced using base-class pointers and via static binding earlier in the program.

Runtime Polymorphism, Virtual Functions and Dynamic Binding

- Let's consider how C++ can implement runtime polymorphism, virtual functions and dynamic binding.
- This will give you a solid understanding of how these capabilities can work.
- More importantly, you'll appreciate the overhead of runtime polymorphism—in terms of additional memory consumption and processor time.
 - This can help you determine when to use runtime polymorphism and when to avoid it.
- C++ standard-library classes generally are implemented without virtual functions to avoid the associated execution-time overhead and achieve optimal performance.

Runtime Polymorphism, Virtual Functions and Dynamic Binding (Cont.)

- First, we'll explain the data structures that the compiler can build to support polymorphism at execution time.
- You'll see that this can be accomplished through **three levels of pointers**—that is, triple indirection.
- Then we'll show how an executing program can use these data structures to execute virtual functions and achieve the dynamic binding associated with polymorphism.
- Our discussion explains a possible implementation.

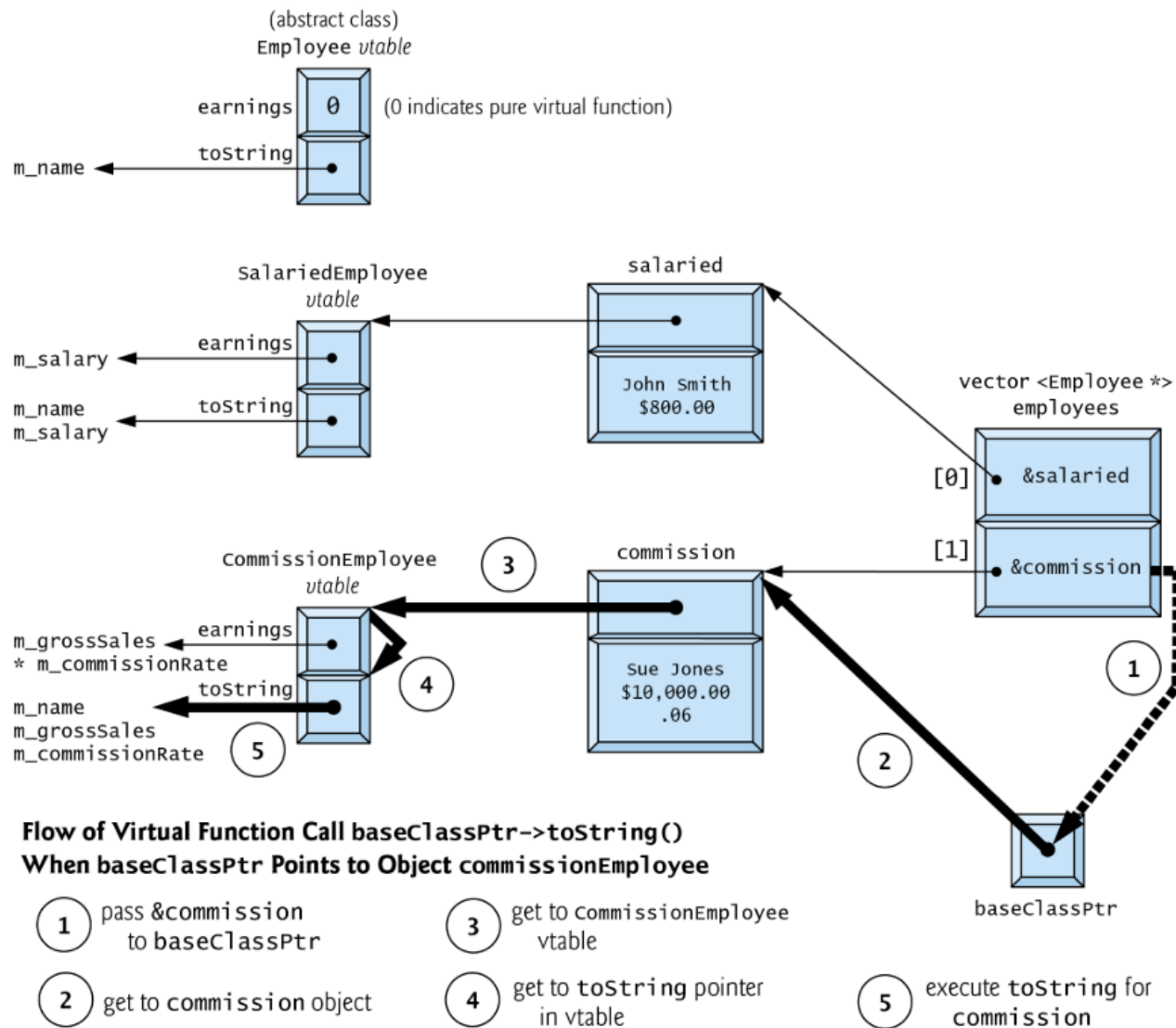
Runtime Polymorphism, Virtual Functions and Dynamic Binding (Cont.)

- Virtual-Function Tables

- When C++ compiles a class with one or more virtual functions, it builds a **virtual function table (vtable)** for that class.
- The vtable contains pointers to the class's virtual functions.
 - ◆ A pointer to a function contains the starting address in memory of the code that performs the function's task.
 - ◆ Just as an array name is implicitly convertible to the address of the array's first element, a function name is implicitly convertible to the starting address of its code.
- **With dynamic binding, an executing program uses a class's vtable to select the proper function implementation** each time a virtual function is called on an object of that class.
- The next figure illustrates the vtables for Employee, SalariedEmployee and CommissionEmployee.

Runtime Polymorphism, Virtual

Fur



Runtime Polymorphism, Virtual Functions and Dynamic Binding (Cont.)

- Employee Class vtable

- The first function pointer in the Employee class vtable is set to 0 (i.e., nullptr) because earnings is a pure virtual function with no implementation.
- The second points to toString, which returns a string containing the employee's name.
- We've abbreviated each toString function's output in this figure to conserve space—note the pointers in the vtables actually point to each function's code.
- Any class with one or more pure virtual functions (represented with the value 0) in its vtable is an abstract class.
- SalariedEmployee and CommissionEmployee are concrete classes—they have no nullptrs in their vtables.

Runtime Polymorphism, Virtual Functions and Dynamic Binding (Cont.)

- SalariedEmployee Class vtable
 - The earnings function pointer in the SalariedEmployee vtable points to that class's override of the earnings function, which returns the salary.
 - SalariedEmployee also overrides toString, so the corresponding function pointer points to SalariedEmployee's toString function, which returns the employee's name and salary.

Runtime Polymorphism, Virtual Functions and Dynamic Binding (Cont.)

- CommissionEmployee Class vtable
 - The earnings function pointer in the CommissionEmployee vtable points to the CommissionEmployee's earnings function, which returns the employee's gross sales multiplied by the commission rate.
 - The toString function pointer points to the CommissionEmployee version of the function, which returns the employee's name, commission rate and gross sales.
 - As in class SalariedEmployee, both functions override class Employee's functions.

Runtime Polymorphism, Virtual Functions and Dynamic Binding (Cont.)

- Inheriting Concrete virtual Functions

- Each concrete class in our Employee case study provides virtual earnings and toString implementations.
 - ◆ You've learned that because earnings is a pure virtual function, each direct derived class of Employee must implement earnings to be a concrete class.
 - ◆ Direct derived classes do not need to implement toString to be considered concrete.
 - They can inherit class Employee's toString implementation.
 - In our case, both derived classes override Employee's toString.
- If a derived class were to inherit toString and not override it, this function's vtable pointer would point to the inherited implementation.
 - ◆ For example, if CommissionEmployee did not override toString, CommissionEmployee's toString function pointer in the vtable would point to the same toString function as in class Employee's vtable.

Runtime Polymorphism, Virtual Functions and Dynamic Binding (Cont.)

- Three Levels of Pointers to Implement Runtime Polymorphism
 - Runtime polymorphism can be accomplished through an elegant data structure involving three levels of pointers.
 - We've discussed one level—**the function pointers in the vtable**.
 - ◆ These point to the functions that execute when a virtual function is invoked.
 - Now we consider the second level of pointers.
 - ◆ Whenever an object with one or more virtual functions is instantiated, the compiler attaches to each object **a pointer to the class's vtable**.
 - ◆ This pointer is usually at the front of the object, but it isn't required to be implemented that way.
 - ◆ In the vtable diagram, these pointers are associated with the SalariedEmployee and CommissionEmployee objects.
 - ◆ The diagram shows each object's data member values.

Runtime Polymorphism, Virtual Functions and Dynamic Binding (Cont.)

- Three Levels of Pointers to Implement Runtime Polymorphism
 - The third level of pointers contains **the addresses of the objects** on which the virtual functions will be called.
 - ◆ The vtable diagram's rightmost column depicts the vector employees containing these Employee pointers.

Runtime Polymorphism, Virtual Functions and Dynamic Binding (Cont.)

- Three Levels of Pointers to Implement Runtime Polymorphism
 - Now let's see how a typical virtual function call executes. Consider in the function `virtualViaPointer` the call `baseClassPtr->toString()`.
 - ◆ Assume that `baseClassPtr` contains `employees[1]`, commission's address in `employees`.
 - ◆ When this statement is compiled, the compiler sees that the call is made via a base-class pointer and that `toString` is a virtual function.
 - ◆ The compiler sees that `toString` is the second entry in each vtable.
 - ◆ So it includes an offset into the table of machine-language object-code pointers to find the code that will execute the virtual function call.

Runtime Polymorphism, Virtual Functions and Dynamic Binding (Cont.)

- Three Levels of Pointers to Implement Runtime Polymorphism
 - The compiler generates code that performs the following operations—the numbers in the list correspond to the circled numbers in the diagram:
 1. Select the *ith* employees entry—the commission object's address—and pass it as an argument to function `virtualViaPointer`.
 - This aims the `baseClassPtr` parameter at the commission object.
 2. Dereference that pointer to access the commission object, which begins with a pointer to class `CommissionEmployee`'s vtable.
 3. Dereference the vtable pointer to access class `CommissionEmployee`'s vtable.
 4. Skip the offset to select the `toString` function pointer.
 5. Execute `toString` for the commission object, returning a string containing the employee's name, gross sales and commission rate.

Runtime Polymorphism, Virtual Functions and Dynamic Binding (Cont.)

- Three Levels of Pointers to Implement Runtime Polymorphism
 - The vtable diagram's data structures may appear complex.
 - ◆ The compiler hides this complexity from you, making runtime polymorphic programming straightforward.
 - ◆ The pointer dereferencing operations and memory accesses for each virtual function call require additional execution time.
 - ◆ The vtables and vtable pointers added to the objects require some additional memory.

Runtime Polymorphism, Virtual Functions and Dynamic Binding.

- Three Levels of Pointers to Implement Runtime Polymorphism
 - Runtime polymorphism is efficient, as typically implemented with virtual functions and dynamic binding in C++.
 - For most applications, you can use these capabilities with nominal impact on execution performance and memory consumption.
 - In some situations, polymorphism's overhead may be too high—such as
 - ◆ in real-time applications with stringent execution-timing performance requirements or
 - ◆ in an application with an enormous number of small objects in which the size of the vtable pointer is large when compared to each object's size.

Program to an Interface, Not an Implementation

- Implementation inheritance is primarily used to define closely related classes with many of the same data members and member function implementations.
 - This kind of inheritance creates tightly coupled classes in which the base class's data members and member functions are inherited into derived classes.
 - Changes to a base class directly affect all corresponding derived classes.

Program to an Interface, Not an Implementation (Cont.)

- Tight coupling can make modifying class hierarchies difficult.
 - Consider modifying the Employee hierarchy to support retirement plans. There are many retirement plan types, including 401(k)s and IRAs.
 - ◆ We might add a pure virtual `makeRetirementDeposit` member function to the class `Employee`.
 - ◆ Then we'd define various derived classes such as `SalariedEmployeeWith401k`, `SalariedEmployeeWithIRA`, `CommissionEmployeeWith401k`, `CommissionEmployeeWithIRA`, etc., each with an appropriate `makeRetirementDeposit` implementation.
 - You can see that you quickly wind up with a proliferation of derived classes, making the hierarchy more challenging to implement and maintain.

Program to an Interface, Not an Implementation (Cont.)

- Small inheritance hierarchies under the control of one person are more manageable than large ones maintained by many people.
 - This is true even with the tight coupling associated with implementation inheritance.

Program to an Interface, Not an Implementation (Cont.)

- Rethinking the Employee Hierarchy: Composition and Dependency Injection
 - Over the years, programmers have written numerous papers, articles and blog posts about the problems with tightly coupled class hierarchies.
 - In this example, we'll refactor our Employee hierarchy so that the Employee's compensation model is not “hardwired” into the class hierarchy.

Program to an Interface, Not an Implementation (Cont.)

- Rethinking the Employee Hierarchy: **Composition and Dependency Injection**
 - To do so, we'll use composition and dependency injection in which a class contains a pointer to an object that provides a behavior required by objects of the class.
 - In our Employee payroll example, that behavior calculates each Employee's earnings.
 - We'll define a new Employee class that has a pointer to a CompensationModel object with earnings and toString member functions.
 - ◆ This Employee class will not be a base class—to emphasize that, we'll make it final.
 - We'll then define derived classes of class CompensationModel that implement how Employees get compensated:
 - ◆ fixed salary and
 - ◆ commission based on gross sales.
 - Of course, we could define other CompensationModels, too.

Program to an Interface, Not an Implementation (Cont.)

- Interface Inheritance Is Best for Flexibility
 - For our CompensationModels, we'll use interface inheritance.
 - ◆ Each CompensationModel concrete class will **inherit from a class containing only pure virtual functions**.
 - Such a class is called an interface or a **pure abstract class**.
 - ◆ The C++ Core Guidelines recommend inheriting from pure abstract classes rather than classes with implementation details.
 - **Interfaces typically do not have data members.**
 - ◆ Interface inheritance may require more work than implementation inheritance because concrete classes must provide both data and implementations of the interface's pure virtual member functions, even if they're similar or identical among classes.
 - ◆ As we'll discuss at the end of the example, this approach gives you additional flexibility by eliminating the tight coupling between classes.
 - ◆ The discussion of device drivers in the context of abstract classes is a good example of how interfaces enable systems to be modified easily.

Program to an Interface, Not an Implementation (Cont.)

- Rethinking the Employee Hierarchy:
CompensationModel Interface

- Let's refactor the previous Employee hierarchy with composition and an interface.
- We can say that each Employee has a CompensationModel.
- The header defines the interface CompensationModel, which is a pure abstract class, so it does not have a .cpp file.

```
1 // CompensationModel.h
2 // CompensationModel "interface" is a pure abstract base class.
3 #pragma once // prevent multiple inclusions of header
4 #include <string>
5
6 class CompensationModel {
7 public:
8     virtual ~CompensationModel() = default; // generated destructor
9     virtual double earnings() const = 0; // pure virtual
10    virtual std::string toString() const = 0; // pure virtual
11};
```

Program to an Interface, Not an Implementation (Cont.)

- Rethinking the Employee Hierarchy:
CompensationModel Interface
 - The class has a compiler-generated virtual destructor and two pure virtual functions:
 - ◆ earnings to calculate an employee's pay based on its CompensationModel, and
 - ◆ toString to create a string representation of a CompensationModel.
 - Any class that inherits from CompensationModel and overrides its pure virtual functions is a CompensationModel that implements this interface.

Program to an Interface, Not an Implementation (Cont.)

- Class Employee: Interface

- Next we define our new Employee class.
- Each Employee has a pointer to the implementation of its CompensationModel (line 16).
- This class is declared final, so it cannot be used as a base class.

```
1 // Employee.h
2 // An Employee "has a" CompensationModel.
3 #pragma once // prevent multiple inclusions of header
4 #include <string>
5 #include <string_view>
6 #include "CompensationModel.h"
7
8 class Employee final {
9 public:
10     Employee(std::string_view name, CompensationModel* modelPtr);
11     void setCompensationModel(CompensationModel* modelPtr);
12     double earnings() const;
13     std::string toString() const;
14 private:
15     std::string m_name{};
16     CompensationModel* m_modelPtr{}; // pointer to an implementation object
17 };
```

Program to an Interface, Not an Implementation (Cont.)

- Class Employee: Implementation

- The constructor (lines 10–11) initializes the Employee's name and aims its CompensationModel pointer at an object that implements the CompensationModel interface.
 - ◆ This technique is known as **constructor injection**.
 - ◆ The constructor receives a pointer (or reference) to another object and stores it in the object being constructed.

```
1 // Employee.cpp
2 // Class Employee member-function definitions.
3 #include <format>
4 #include <string>
5 #include "CompensationModel.h"
6 #include "Employee.h"
7
8 // constructor performs "constructor injection" to initialize
9 // the CompensationModel pointer to a CompensationModel implementation
10 Employee::Employee(std::string_view name, CompensationModel* modelPtr)
11     : m_name{name}, m_modelPtr{modelPtr} {}
```

Program to an Interface, Not an Implementation (Cont.)

- Class Employee: Implementation

- The setCompensationModel member function (lines 15–17) enables the client code to change an Employee's CompensationModel by aiming m_modelPtr at a different CompensationModel object.
 - ◆ This technique is known as [property injection](#).

```
13 // set function performs "property injection" to change the
14 // CompensationModel pointer to a new CompensationModel implementation
15 void Employee::setCompensationModel(CompensationModel* modelPtr) {
16     m_modelPtr = modelPtr;
17 }
```

Program to an Interface, Not an Implementation (Cont.)

- Class Employee: Implementation

- The earnings member function (lines 20–22) determines the Employee's earnings by calling the CompensationModel implementation's earnings member function via the CompensationModel pointer m_modelPtr.
- The toString member function (lines 25–27) creates an Employee's string representation.
 - ◆ This consists of the Employee's name, followed by the compensation information we get by calling the CompensationModel implementation's toString member function via m_modelPtr.

```
19 // use the CompensationModel to calculate the Employee's earnings
20 double Employee::earnings() const {
21     return m_modelPtr->earnings();
22 };
23
24 // return string representation of Employee object
25 std::string Employee::toString() const {
26     return std::format("{}\n{}", m_name, m_modelPtr->toString());
27 }
```

Program to an Interface, Not an Implementation (Cont.)

- Class Employee: Implementation
 - **Constructor injection** and **property injection** are both forms of **dependency injection**.
 - ◆ **You specify part of an object's behavior by providing a pointer or reference to an object that defines the behavior.**
 - ◆ In this example, a CompensationModel provides the behavior that enables an Employee to calculate earnings and generate a string representation.

Program to an Interface, Not an Implementation (Cont.)

- CompensationModel Implementations
 - Next, let's define our CompensationModel implementations.
 - Objects of these classes will be injected into Employee objects to specify how to calculate their earnings.

Program to an Interface, Not an Implementation (Cont.)

- Salaried Derived Class of CompensationModel
 - A Salaried compensation model defines how to pay an Employee who receives a fixed salary.
 - ◆ The class contains an m_salary data member and overrides interface CompensationModel's earnings and toString member functions.
 - ◆ Salaried is declared final (line 7), so it is a leaf node in the CompensationModel hierarchy.
 - It may NOT be used as a base class.

```
1 // Salaried.h
2 // Salaried implements the CompensationModel interface.
3 #pragma once
4 #include <string>
5 #include "CompensationModel.h" // CompensationModel definition
6
7 class Salaried final : public CompensationModel {
8 public:
9     explicit Salaried(double salary);
10    double earnings() const override;
11    std::string toString() const override;
12 private:
13    double m_salary{0.0};
14 };
```

Program to an Interface, Not an Implementation (Cont.)

- Salaried Derived Class of CompensationModel
 - Implementation of Salaried compensation model

```
1 // Salaried.cpp
2 // Salaried compensation model member-function definitions.
3 #include <format>
4 #include <stdexcept>
5 #include "Salaried.h" // class definition
6
7 // constructor
8 Salaried::Salaried(double salary) : m_salary{salary} {
9     if (m_salary < 0.0) {
10         throw std::invalid_argument("Weekly salary must be >= 0.0");
11     }
12 }
13
14 // override CompensationModel pure virtual function earnings
15 double Salaried::earnings() const {return m_salary;}
16
17 // override CompensationModel pure virtual function toString
18 std::string Salaried::toString() const {
19     return std::format("salary: ${:.2f}", m_salary);
20 }
```

Program to an Interface, Not an Implementation (Cont.)

- Commission Derived Class of CompensationModel
 - A Commission compensation model defines how to pay an Employee commission based on gross sales.
 - ◆ The class contains data members m_grossSales and m_commissionRate and overrides interface CompensationModel's earnings and toString member functions.
 - ◆ Like class Salaried, class Commission is declared final (line 7), so it is a leaf node in the CompensationModel hierarchy. It may not be used as a base class.

```
1 // Commission.h
2 // Commission implements the CompensationModel interface.
3 #pragma once
4 #include <string>
5 #include "CompensationModel.h" // CompensationModel definition
6
7 class Commission final : public CompensationModel {
8 public:
9     Commission(double grossSales, double commissionRate);
10    double earnings() const override;
11    std::string toString() const override;
12 private:
13    double m_grossSales{0.0};
14    double m_commissionRate{0.0};
15 };
```

Program to an Interface, Not an Implementation (Cont.)

- Commission Derived Class of CompensationModel
 - Implementation of Commission compensation model

```
1 // Commission.cpp
2 // Commission member-function definitions.
3 #include <format>
4 #include <stdexcept>
5 #include "Commission.h" // class definition
6
7 // constructor
8 Commission::Commission(double grossSales, double commissionRate)
9     : m_grossSales{grossSales}, m_commissionRate{commissionRate} {
10
11     if (m_grossSales < 0.0) {
12         throw std::invalid_argument("Gross sales must be >= 0.0");
13     }
14
15     if (m_commissionRate <= 0.0 || m_commissionRate >= 1.0) {
16         throw std::invalid_argument(
17             "Commission rate must be > 0.0 and < 1.0");
18     }
19 }
20
21 // override CompensationModel pure virtual function earnings
22 double Commission::earnings() const {
23     return m_grossSales * m_commissionRate;
24 }
25
26 // override CompensationModel pure virtual function toString
27 std::string Commission::toString() const {
28     return std::format("gross sales: ${:.2f}; commission rate: {:.2f}",
29         m_grossSales, m_commissionRate);
30 }
```

Program to an Interface, Not an Implementation (Cont.)

- Testing the New Hierarchy
 - We've created our CompensationModel interface and derived-class implementations defining how Employees get paid.
 - Now, let's create Employee objects and initialize each with an appropriate concrete CompensationModel implementation.

```
1 // main.cpp
2 // Processing Employees with various CompensationModels.
3 #include <format>
4 #include <iostream>
5 #include <vector>
6 #include "Employee.h"
7 #include "Salaried.h"
8 #include "Commission.h"
9
10 int main() {
```

Program to an Interface, Not an Implementation (Cont.)

- Testing the New Hierarchy

- Line 12 creates a Salaried compensation model object (salaried).
- Line 13 creates the Employee salariedEmployee and injects its CompensationModel, passing a pointer to salaried as the second constructor argument.
- Line 15 creates a Commission compensation model object (commission).
- Line 16 creates the Employee commissionEmployee and injects its CompensationModel, passing a pointer to commission as the second constructor argument.

```
11 // create CompensationModels and Employees
12 Salaried salaried{800.0};
13 Employee salariedEmployee{"Pierre Simon", &salaried};
14
15 Commission commission{10000, .06};
16 Employee commissionEmployee{"Sierra Dembo", &commission};
```

Program to an Interface, Not an Implementation (Cont.)

- Testing the New Hierarchy

- Line 19 creates a vector of Employees and initializes it with the salariedEmployee and commissionEmployee objects.
- Finally, lines 22–25 iterate through the vector, displaying each Employee's string representation and earnings.

```
18 // create and initialize vector of Employees
19 std::vector employees{salariedEmployee, commissionEmployee};
20
21 // print each Employee's information and earnings
22 for (const Employee& employee : employees) {
23     std::cout << std::format("{}\nearned: {:.2f}\n\n",
24                             employee.toString(), employee.earnings());
25 }
26 }
```

Pierre Simon
salary: \$800.00
earned: \$800.00

Sierra Dembo
gross sales: \$10000.00; commission rate: 0.06
earned: \$600.00

Program to an Interface, Not an Implementation (Cont.)

- Dependency Injection Design Benefits
 - Flexibility If CompensationModels Change
 - Flexibility If Employees Are Promoted
 - Flexibility If Employees Acquire New Capabilities

Program to an Interface, Not an Implementation (Cont.)

- Flexibility If CompensationModels Change
 - Declaring the CompensationModels as separate classes that implement the same interface provides flexibility for future changes.
 - Suppose a company adds new ways to pay employees. We can simply define a new CompensationModel derived class with an appropriate earnings function.

Program to an Interface, Not an Implementation (Cont.)

- Flexibility If Employees Are Promoted

- The interface-based, composition-and-dependency-injection approach used in this example is more flexible than previous class hierarchy.
- In previous example, if an Employee were promoted, you'd need to change its object type by creating a new object of the appropriate Employee derived class, then moving data into the new object.
- **Using dependency injection, you can call Employee's setCompensationModel member function and inject a pointer to a different CompensationModel that replaces the existing one.**

Program to an Interface, Not an Implementation (Cont.)

- Flexibility If Employees Acquire New Capabilities
 - The interface-based composition and dependency-injection approach is also more flexible for enhancing class Employee.
 - If we decide to support retirement plans (such as 401(k)s and IRAs), we could say that every Employee has a RetirementPlan.
 - We'd define the RetirementPlan interface with a makeRetirementDeposit member function and provide appropriate derived-class implementations.

Program to an Interface, Not an Implementation.

- Flexibility If Employees Acquire New Capabilities
 - Using interface-based composition and dependency injection, as shown in this example, requires only small changes to class Employee to support RetirementPlans:
 - ◆ a data member that points to a RetirementPlan,
 - ◆ one more constructor argument to initialize the RetirementPlan pointer and
 - ◆ a setRetirementPlan member function we can call if we ever need to change the RetirementPlan.